

Real-time Target Speaker Extraction

Grant Booysen

25849646@sun.ac.za

Thesis presented in partial fulfilment of the requirements for the degree of
MSc in Machine Learning & Artificial Intelligence
in the Faculty of Science at Stellenbosch University

Supervisors: DJ Swanevelder and David Botha (SmartOperator)

November 2026

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Grant Booysen

November 2026

Signature

Date

Abstract

Acknowledgements

Contents

Declaration	i
Abstract	ii
Acknowledgements	iii
Nomenclature	vi
1. Introduction	1
2. Literature Review	2
2.1. Target speaker extraction	2
2.2. Streaming separation	3
2.3. How extraction is evaluated	3
2.4. Speech LLMs as listeners and judges	4
3. Methodology	5
3.1. Data construction	5
3.1.1. Why the data is constructed	5
3.1.2. Source of audio	5
3.1.3. Trial construction	6
3.2. Baseline model architecture	10
3.2.1. Band-split Recurrent Neural Network (BSRNN)	10
3.2.2. Short-time Fourier transform	10
3.2.3. Band plan	11
3.2.4. Per-band normalisation	12
3.2.5. Residual LSTM block	13
3.2.6. Band and sequence model	14
3.2.7. Estimation module	15
3.2.8. Time-frequency map	16
3.2.9. Objective function	17
3.3. Extension model architecture	21
3.3.1. Splitting the cue into parts	21
3.3.2. Speaker context embedding	22
3.4. Metrics	23
3.4.1. Trial definitions	23
3.4.2. Text normalisation	24
3.4.3. Live-model Content Fidelity Word Error Rate	24

3.4.4.	Interferer Content Rate	24
3.4.5.	Fabrication Rate	26
3.4.6.	Deep Noise Suppression Mean Opinion Score	26
3.4.7.	Signal to Distortion/Interference/Artefact Ratios	27
3.4.8.	Anchors	27
4.	Experiments	28
4.1.	Training setup	28
4.1.1.	Data Augmentation	28
4.1.2.	Optimisation and hardware	28
4.1.3.	Model Selection	28
4.1.4.	Systems compared	29
4.1.5.	Training curves	29
4.2.	Evaluation setup	29
4.2.1.	Listeners	29
4.2.2.	Latency measures	32
5.	Results	32
5.1.	Content fidelity	32
5.1.1.	The live judge - Gemini Live	33
5.1.2.	The offline transcriber	35
5.2.	Error composition	36
5.3.	Signal-domain separation	37
5.4.	Perceptual quality	38
5.5.	Latency and throughput	39
6.	Conclusion	39
6.1.	Summary of contributions	39
6.2.	Limitations and future work	39
	References	39
A.	Evaluation protocol details	45
A.1.	Prompt for live-model transcription	45
A.2.	Pinned components of the measuring instrument	45
B.	Room construction	45
C.	Construction parameters	46

Nomenclature

Abbreviations

AMI	Augmented multi-party interaction
ASR	Automatic speech recognition
BAK	Background intrusiveness (DNSMOS P.835 sub-score)
BSRNN	Band-split recurrent neural network
CARTSE	Li and Seki's online REAL-TSE system [1]
DNSMOS	Deep noise suppression mean opinion score
EQ	Equalisation
FR	Fabrication rate
GLU	Gated linear unit
ICR	Interferer content rate
IHM	Individual headset microphone
ITU-T	ITU Telecommunication Standardization Sector
LAM	Large audio model
LCF-WER	Live-model content fidelity word error rate
LSTM	Long short-term memory
LUFS	Loudness units relative to full scale
MLP	Multilayer perceptron
NLTK	Natural Language Toolkit
OVRL	Overall quality (DNSMOS P.835 sub-score)
REAL-TSE	Real-world target speaker extraction (challenge)
RIR	Room impulse response
RTF	Real-time factor
SAR	Signal-to-artefacts ratio
SDR	Signal-to-distortion ratio
SIG	Speech signal quality (DNSMOS P.835 sub-score)
SIR	Signal-to-interference ratio
SI-SDR	Scale-invariant signal-to-distortion ratio
SLT	IEEE Workshop on Spoken Language Technology
SNR	Signal-to-noise ratio
STFT	Short-time Fourier transform
TF Map	Time-frequency map
TSE	Target speaker extraction
VAD	Voice activity detection
WER	Word error rate

Symbols

Signals and transcriptions

x	Mixture signal
s	Target reference signal
$\hat{s}$	Model output, the estimated target
d	Interfering speech signal
$y(\cdot)$	Transcription of the subscripted signal
y_s^*	Ground-truth LibriSpeech transcription of the target

Model and objective

X, X_t	STFT of the mixture; frame t of it
X_e, x_e	STFT of the enrollment; the enrollment waveform
$S, \hat{S}$	STFT of the target; of the model output
$N, H, h[n]$	STFT window length, hop and window function
$\tilde{x}_t, \tilde{e}_j$	Unit-length mixture frame; unit-length enrollment frame
$C_{t,j}$	Cosine similarity of mixture frame t and enrollment frame j
$A_{t,j}$	Softmax weight of enrollment frame j for mixture frame t
μ_t, u_t	Blended enrollment template; its unit-length direction
m_t, ϕ_t	Projection of $ X_t $ onto u_t ; the baseline cue $m_t u_t$
c_t, r_t	Similarity of $ X_t $ to u_t ; residual $ X_t - m_t u_t$
M, R	Complex mask; residual spectrogram
α, s_{proj}	Gain of the output along the target; αs
$e_{\text{int}}, e_{\text{art}}$	Interference (other speaker and noise); artefact
$L_{\text{pres}}, L_{\text{MR}}, L_{\text{gain}}, L_{\text{abs}}$	Loss terms
$w, w_{\text{m}}, w_{\text{g}}$	Loss weights
$\tau_{\text{pres}}, \tau_{\text{abs}}$	Floors of L_{pres} (10^{-3}) and L_{abs} (10^{-2})

1. Introduction

Voice assistants engage with users through direct, live speech-to-speech models such as Gemini Live [2]. With the appearance of more voice based applications, the ability to engage with these tools in acoustically challenging environments is becoming increasingly important. A particular problem is the presence of interfering speech, which is misinterpreted by the live speech models as the intended target user speech. Interfering speech thus will confuse the downstream model as to the intended user input, which will lead to a poor user experience. Secondly, live speech models are exposed to noisy environments that include reverberation, background audio and other acoustic challenges. When presented to these conditions, live models are seen to interpret the audio as actual speech and thus invent words. The hallucination again negatively impacts the user experience as the interpretation from the model of the user's input is not what the user intended. The work in this report aims to address these issues through the development of a target speaker extractor that aims to preprocess the audio input that is given to the live model. The extractor is constructed under the following constraints: it must be causal, process audio in real time and operate on a single channel of audio. However, an extractor can itself introduce artefacts that the live model hears as words, so it is not enough to measure whether the interfering speaker was removed; the evaluation must show which of the two errors the extractor causes.

Under these constraints, the contribution of this work lies not in a new architecture but in how the extractor is evaluated. The novel contributions are:

1. **A live-model content fidelity evaluation suite.** The live model transcribes the extractor's output, and its errors are split into words leaked from the interfering speaker and words that neither speaker said (Section 3.4).
2. **Evidence that the metrics are needed.** A single word error rate hides changes that the split reveals, and an offline transcriber misjudges changes that the live model does not notice (Section 5).

To produce and test these, the following was built:

3. **A constructed dataset and evaluation protocol.** 9955 training trials from 1172 LibriSpeech speakers, each rendered by constructing a simulated room with a random reverberation time and microphone and speaker positions, and then adding real background noise. Each speaker has exact transcripts. The dataset includes trials where the target is absent so that the extractor learns to stay silent (Section 3.1).
4. **A streaming target speaker extractor.** The REAL-TSE baseline architecture [3], implemented for this work and made fully causal, with the enrollment cue split into parts and a speaker embedding added (Section 3.2, Section 3.3). It runs in real time on a laptop

CPU within the latency budget and reduces Gemini’s word error rate from 55.2 % for the baseline to 39.6 %, while halving the share of the interfering speaker’s words that reach Gemini. In cases where there is only interfering speech, it stays silent five times as often as WeSep, a larger, established, offline system trained on different data.

5. **A training loop with a proxy objective.** The live model cannot be differentiated against, thus the extractor is trained on a signal-level proxy objective adapted from CARTSE [1].

All results are for two-speaker mixtures and are optimised for Gemini.

The remainder of this report is structured as follows: Section 2 presents and reviews target speaker extraction, streaming separation, how extraction is typically measured, and speech models as listeners. Section 3 describes the process of generating the data, the architecture of the extractor and the metrics. Section 4 sets out how the model is trained, selected and evaluated. Section 5 presents the results of the evaluation and Section 6 concludes the report with a summary and potential future work.

2. Literature Review

2.1. Target speaker extraction

Target speech extraction (TSE) is already a well established task in the speech processing literature. With the introduction of listening devices such as meeting transcription systems, hearing aids, and smart speakers, these devices benefit from the ability for more robust speech recognition in the presence of background noise and competing speakers. The main idea behind the task is best described by Cherry’s cocktail party problem [4]. Human’s have the innate ability to focus on a single speaker in noisy environments, and using various indicators such as visual, spatial, and acoustic cues, human’s can understand the speech of a target speaker while ignoring other speakers. The survey done by Žmolíková et al. [5] provides a comprehensive overview of the task, differentiating between three main tasks: target speaker extraction, blind source separation, and noise suppression. In this work, the focus is on target speaker extraction using only voice samples as an acoustic cue.

The intention behind the TSE task is to be able to deploy a system into a real-world scenario. However due to the variability in the scenarios that a TSE system may be deployed in, a system must not necessarily be trained to only extract a speaker from among a noisy environment of other speakers. The survey by Žmolíková et al. [5] in 2023 highlights the gap that many TSE systems are not trained to go silent when the target speaker is not present, which is a separate task to extraction. The systems have thus assumed to always have a target present in the mixture, which confounds the model to assume that the target speaker is present whenever the mixture is processed. Another issue raised by Žmolíková et al. [5] is that the use of signal level metrics such as signal-to-distortion ratio (SDR) are indicative of the ability of the model

to extract speech, but (i) it does not correlate with audible quality from the perspective of a human listener, and (ii) it does not reflect the ability of the model to select or identify a target speaker as an evaluation metric. Among other issues, the a final point to raise that is included in the survey is that TSE systems are being moved to low latency streaming setups. Thus it is important to consider lightweight architectures that can be deployed in real-time scenarios.

2.2. Streaming separation

Separation models work with either a raw waveform or on a spectrogram. Some architectures such as Conv-TasNet [6] favoured for waveform with the reasoning that calculating a spectrogram contributes to the latency of the system. However, Yu et al. adapted a Band-Split RNN (BSRNN) architecture originally designed for music as an offline model [7] to speech enhancement tasks with an online configuration [8] which Žmolíková et al. [5] list as another name for TSE. Yu et al. found that their online implementation ran faster than real-time on an Intel i5 CPU.

TSE systems must work frame by frame, or chunk by chunk. The REAL-TSE challenge set a benchmark to say that a system must run with an algorithmic latency of 100 ms or less [3]. Algorithmic latency is the delay built in the model. In other words, how long the system has to wait for audio before it can produce an output. Secondly, the architectures need to be constructed in a causal manner. For the REAL-TSE this included swapping bidirectional LSTMs for unidirectional LSTMs in the time domain, and secondly changing the normalisation strategies to work over a single frame or chunk rather than over the entire sequence [3]. A key finding from the REAL-TSE challenge is that a claimed and measured latency often disagreed. For true transparency on the latency of a system the recommendation is to have a component breakdown of the algorithmic latency. This work does this and furthermore highlights the processing time measured by real-time factor (RTF).

A concern about streaming is that it makes it harder to identify the target speaker. As found in the survey by Žmolíková et al. [5], typical TSE systems use the entire context of an audio clip to identify the target speaker. However, in a streaming scenario, the system must identify quite quickly speaker identity. Finally in this work, the assumption that a 200 ms to 300 ms budget is sufficient that reflects a conversational use case.

2.3. How extraction is evaluated

When making use of signal based metrics, practitioners are often required to use simulated data. This is because a clean reference signal is required for comparison against the model output. By nature it is quite difficult to obtain perfectly clean speech in a real-world scenario. Popular resources to use for simulated mixtures include Libri2Mix [9] and WSJ0-2mix [10]. Thus the first issue related to evaluation is the translation of performance on simulated data to real-world data. Zhang et al. [11] show that there is not necessarily a direct translation in performance: in

multi-channel speech enhancement, the model with the best signal-to-distortion ratio (SDR) on simulated data more than doubled its word error rate (WER) compared of the unprocessed input on real recordings from 19.5 % to 41.5 %. Signal scores also fail in the following manner for TSE: (i) extracting a wrong speaker can still yield a low signal score due to similarities in speech, and (ii) a silent target leaves the score undefined.

A natural guess to evaluate the performance of a TSE system is to suggest using perceptual metrics. These metrics are designed to mimic what a human listener would perceive when listening to the audio, for example DNSMOS [12]. However, as found by Wang et al. [3], these metrics can be gamed by adding adversarial perturbations to the waveform to inflate it. In other words, an extractor can be trained to maximise the perceptual metric without it improving the actual quality of the audio. Secondly, these perceptual metrics still fail in TSE as they do not identify if the wrong person is speaking.

The next progression would be to transcribe the audio and compare it against a ground truth transcription. The standard approach is to use the simple word error rate (WER) metric. However, this metric hides nuance in the failure of an extractor. Recognisers are hurt by the distortions of the extractor rather than by leftover noise left in the audio [13] and secondly, a different recogniser may have different performance on the same audio. This leads into the shortcomings of WER as a metric for TSE. Suppose a word is heard from the interfering speaker, and secondly another word that neither speaker said and was invented, both can appear in the metric as a substitution or an insertion error. Finally the WER is only defined if there is target speech present.

2.4. Speech LLMs as listeners and judges

It is not a novel idea to make use of large language models (LLMs) to evaluate the quality of speech. In fact, Manakul et al. [14] found that large audio models (LAMs), which are LLMs that accept audio as input, are able to rank speech systems similarly to human raters. However, for tasks related to measuring speaking rate or speaker identity tasks, the performance of these models is poor and often are close to random chance. The authors also found that these established LAMs that they tested are robust against noise and are more reliable on lexical content as opposed to non-lexical content.

In Manakul et al. the study uses the model as a rater. The LAM is prompted for a verdict, such as preference over audio samples, and the final decision becomes the score. This work uses the model differently. The live model is only asked to write down what it heard and the score is computed by comparing the transcription to a ground truth text (Section 3.4.3). Each clip that is presented to the LLM is on its own, which mitigates positional bias that was found by Manakul et al. Their noise robustness was also only tested using white noise and did not explore competing speakers. Using the model as a listener rather than a rater is less common. The closest work reviewed here, Fela and Mowlae [15], found that standard audio-quality metrics

fail to predict how often speech enhancement changes an LLM’s downstream output. This is a third reason for the revised evaluation suite as discussed in the Section 2.1 and Section 2.3.

3. Methodology

3.1. Data construction

The section describes the construction of the dataset used for this work. It includes a description of the necessity of constructing the data from scratch, the source of the audio used, and the construction of the trials.

3.1.1. Why the data is constructed

There are three main considerations for the dataset that need to be met: (i) training requires a clean target signal as a reference for what the extractor should output, (ii) the data must possess exact transcriptions of the target signal to allow for transcription-based evaluation, and (iii) the data must possess both best case and worst case scenarios in order to interpret the performance of the extractor relative to these scenarios. These considerations limit the available choices of publically available datasets.

Constructing the data is also the established practice. The 2026 REAL-TSE challenge [3] supplied no training set at all, and required that participants document whichever open-source data they used. The organisers after reviewing the submissions, describe the data recipe that the entrants converged on:

“Most teams synthesized mixtures from publicly available speech corpora such as LibriSpeech...; added noise from WHAM!...; and introduced reverberation with measured/simulated RIRs, while varying the number of speakers, SNR/SIR, RT60, chunk length, and enrollment quality. Many teams also injected target-absent or noise-only segments, forcing systems not only to extract the target when present but also to suppress non-target speech and avoid false extraction.” [3]

The dataset constructed here follows that recipe, and the sections below describe each of its components in turn. The organisers further observed that several of the strongest entries were built on backbones near-identical to the official baseline architecture, the BSRNN, yet scored substantially better. They attribute the success to data simulation, real-data adaptation, pseudo-label filtering, loss design and post-processing rather than to architecture alone.

3.1.2. Source of audio

The audio data was collected from a variety of sources to produce a diverse dataset. Speech was chosen for target and interferer from the LibriSpeech [16] dataset, which is a corpus of read

English speech derived from audiobooks. The background noise was chosen from WHAM! [17] which is a dataset of real-world noise recordings considering environments such as offices, streets, and cafés.

3.1.3. Trial construction

The trials are structured in a way to replicate the conditions of a real-world target speaker extraction scenario. Due to the difficulty of the TSE task, it was decided that the data should reflect a variety of conditions, but primarily focus on the base case task of a target speaker close to a recording device with an interfering speaker in a similar proximity and faint background noise. The difficulty came in constructing the dataset to be realistic enough to simulate a real-world scenario, but also not so difficult that a simple extractor of limited capacity would be unable to learn the task. A visual representation of the trial construction can be found in Figure 3.1.

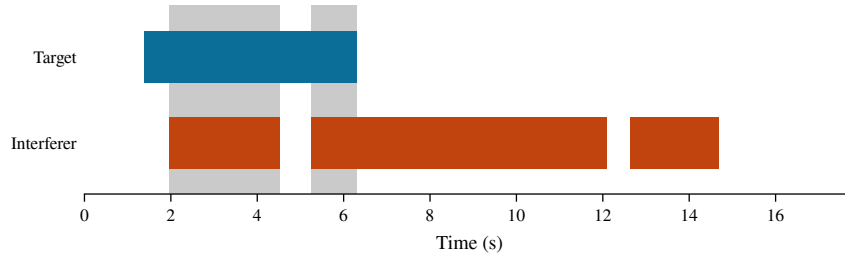


Figure 3.1: Visual representation of the trial construction. The target and interferer are drawn from LibriSpeech, and the noise bed is drawn from WHAM!. The shaded areas show the overlapping regions of speech.

A single trial consists of the following components: (i) target speech, (ii) interferer speech, (iii) noise bed, (iv) the mixture of the first three components, and (v) an enrollment clip of the target speaker. Each trial falls into one of four cases: *Case 1*, both speakers present; *Case 2*, the target alone; *Case 3*, the other speaker alone; and *Case 4*, noise only. The mixture is constructed to be 15-20 seconds long. The trials are drawn from two separate regimes: a base case and a difficult case. The base case is constructed to be a simple but realistic scenario, and difficult cases are edge cases that are constructed to be more challenging for the extractor. There both male and female speakers in the dataset. To simulate room acoustics, including reverberation, room dimensions, source and microphone positions, the `pyroomacoustics` [18] package is used to simulate the room impulse response. The trials are constructed to be as realistic as possible, but also to be reproducible and controllable. A sample of the room construction can be found in the Appendix B.

Enrollment. The enrollment clip is a short audio segment of the target speaker. The clip is used to provide the extractor with a reference of what the target speaker sounds like. The clips are taken from the same source as the target speech, which is LibriSpeech. To avoid bias

in the extractor, the 5 s enrollment clip is first attempted to be drawn from a different book reading. If this is unsuccessful, then the clip is drawn from a different chapter of the same book. Finally, if this is also unsuccessful, then the clip is drawn from a different utterance of the same chapter. The reason for this is to avoid the extractor memorising the target speech by learning to recognise the content rather than the *manner* that the target speaker speaks. The distribution of enrollment clips across the dataset is shown in Figure 3.2. Again to avoid bias, the enrollment clips are simulated to come from different recording devices. This is done by applying equalisation (EQ) augmentation to the enrollment clips by applying a random EQ filter to the clip with a probability of 0.5. This technique was suggested by the CARTSE [1] submission for the 2026 REAL-TSE challenge [3].

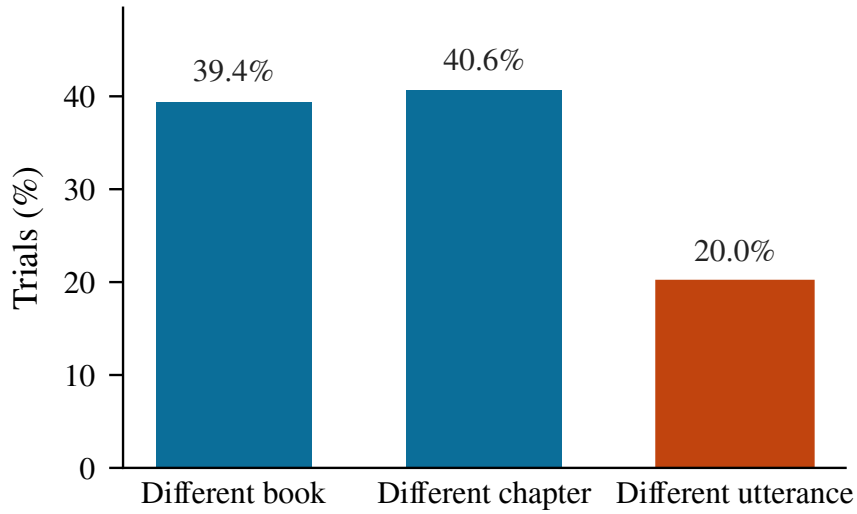


Figure 3.2: Distribution of the source of the enrollment clips across the dataset.

Parameter distribution choice. The Signal-to-Interference Ratio (SIR) is chosen uniformly from the range of -10 to 10 dB, and the Signal-to-Noise Ratio (SNR) is chosen uniformly from the range of 0 to 20 dB. These two ratios simulate the relative loudness of the target speaker to the interferer and background noise, respectively. A negative number for the SIR indicates that the interferer is louder than the target speaker, and the lower the SNR, the louder the background noise is relative to the target speaker. The SIR was chosen to be symmetric around 0 dB after experimentation found that if the target was always (or majority of the time) louder than the interferer, then the extractor would just learn to follow the speech that was loudest and hence ignore the conditioning of the enrollment audio. This defeats the purpose of the task hence the stated requirement.

The reverberation of each room is set by a reverberation time, T_{60} , which is defined to be the time it takes for sound to decay by 60 dB. The larger the value the more reverberant the room is. T_{60} is drawn from a uniform distribution between 0.25 s to 0.6 s, which are values that span a carpeted office to a larger room with hard surfaces. To enforce realism, the absorption rate of the walls is also considered. The absorption rate is considered using the Sabine relation [19].

The relation connects the physical properties of a room to its volume, its surface area and the absorption of its surfaces, so once the dimensions and the T_{60} have been drawn the absorption is fixed by them and is solved for rather than sampled. A draw is rejected and replaced if the requested T_{60} is not achievable in a room of the drawn size, which is also the reason for the lower bound of 0.25 s.

The remaining parameters and their distributions can be found in the Appendix C

Target-absent trials. The extractor needs example trials to learn to rely on the conditioning of the enrollment audio to identify target speech. The target-absent trials support this idea by having the model learn to stay silent when the enrollment speaker is not present. The split is as follows for these trials: 49.5% both speakers present, 25.5% target only, 19.6% interferer only and 5.4% noise only.

Splits. The dataset is split into a training and validation sets. Each of these sets have their own distinct set of speakers such that there is no overlap of speakers between the sets. The training set consists of 9955 trials, consisting of 1172 unique speakers. The idea behind this is to ensure that the model does not memorise speakers, but rather to learn to extract the target speaker based on the enrollment clip. The training split was progressively built up in stages from 1989 to 4976 and then finally to 9955 trials. This was done to measure the effect of the quantity of data on the performance of the extractor. The validation set to consists of 103 trials where both speakers are present and did not include the other cases. The evaluation holdout set consists of 200 trials of 40 unique speakers that is comprised of the following cases: (i) 103 trials where both speakers are present, (ii) 47 trials of the target speaker alone, (iii) 42 trials of the interfering speaker alone and (iv) 8 trials of neither speaker present.

Limitations. The dataset is constructed to be as realistic as possible, but it is still a simulated dataset. The dataset is restricted to two speakers, which can confound the extractor to expect the target to be present if two speakers are present. Secondly, the combined audio forms *read* speech and not *conversational* speech. This means that the resulting audio does not have natural turn-taking, or conversational dynamics. These dynamics are simulated through the overlap of the target and interferer, but it is not a perfect simulation. Next, the dataset is restricted to English speech, and shoebox rooms only. Finally, the noise beds use are from WHAM! [17] and range in length from 5 to 20 seconds long. This means that the noise beds are not long enough occasionally to cover the entire trial. To solve this, the noise bed is looped to cover the entire trial.

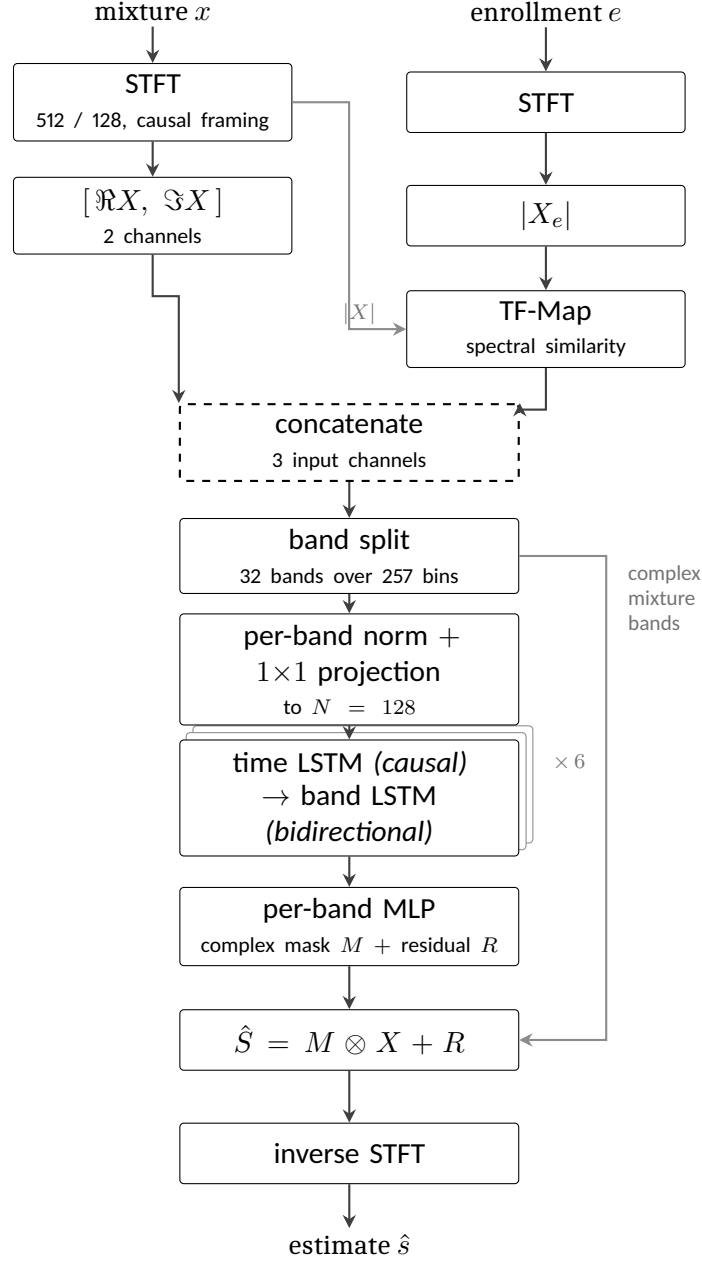


Figure 3.3: The extractor as implemented. Band splitting and the interleaved band-level and sequence-level modelling follow [7]; the mask-plus-residual estimator follows [8]; TF-Map conditioning follows [20]. The causal time path and the third input channel are this work’s.

3.2. Baseline model architecture

3.2.1. Band-split Recurrent Neural Network (BSRNN)

Luo and Yu [7] propose the BSRNN model as a frequency-domain separation model that is designed for high sample rate audio, in which the input audio is transformed from its waveform representation into a time-frequency representation using the Short-Time Fourier Transform (STFT). The model then divides the input spectrum into multiple frequency bands that are modelled independently. The original use case for the BSRNN model is music source separation, but in this work, the model is adapted to allow for the extraction of a target speaker from a mixture of speech signals. The development from music source separation to the method of the target speaker extraction (TSE) approach that this work follows came from the following lineage: (i) music source separation was adapted to personalised speech enhancement (PSE) by Yu et al. [8]. The authors proposed target speaker extraction in the PSE task by appending a target speaker enrollment module that was designed to suppress any interference. PSE can be summarised as enrollment-conditioned extraction. (ii) Zhang et al. followed this framework and proposed a new module for the enrollment conditioning module, namely the Time-Frequency Map (TF Map) [20].

3.2.2. Short-time Fourier transform

When speaking, the audio contains energy. The energy is not evenly distributed across frequencies, and in normal speech it is not evenly distributed across time either. The energy is concentrated in particular frequency bands by the unique resonances of the speaker's vocal tract, and is also concentrated in time by syllables, words and phrases. Audio models therefore want to leverage that structure, and require the input to be presented in a representation that allows for those concentrations to be observed. The waveform itself cannot represent that structure as it is a one-dimensional signal and is not aligned with the human perception of sound.

The short-time Fourier transform (STFT) produces that representation. The waveform is cut into short overlapping segments. These segments are used to compute the Fourier transform, which is used to decompose the signal into its frequency components. These overlapping segments are called frames, and the idea is to pass each of these segments through the Fourier transform by first multiplying the segment by a window function. The window function, which is typically a Hann window, tapers at the ends such that when multiplied, silences the ends of the segment to isolate the middle. After this a discrete Fourier transform is taken of each segment. The result is one complex number per frequency bin per segment: for a real input x , the coefficient at frequency bin f and frame t is

$$X(f, t) = \sum_{n=0}^{N-1} x[tH + n] h[n] e^{-j2\pi fn/N}, \quad (3.1)$$

where j is the imaginary unit ($j = \sqrt{-1}$), N is the window length in samples, H is the hop by which successive windows advance, and $h[n]$ is the window function. The hop is what makes the segments overlap: consecutive frames share $N - H$ samples, so a feature that falls near the edge of one frame sits comfortably inside the next. For the implementation in this work, a window size of $N = 512$ is selected, with a hop of $H = 128$ samples at 16 kHz gives a 32 ms window shifting by 8 ms. These are the values Yu et al. [8] specify for their 16 kHz model. Secondly given the constraint of the streaming budget of 200 ms to 300 ms, the framing consumes well under a quarter of that budget. Finally, the approach in this work forms the frames without centring creating the causal framing. The reason for this is that centring each frame on its own timestamp makes frame t consume $N/2 = 256$ samples – 16 ms – of audio recorded *after* t . Since the objective is to create an online, streaming system and thus the model should not depend on future input. To account for the beginning samples of the audio, the signal is padded by $N - H$ samples at the start, which is a kind of cold start of a live system whose input buffer begins empty.

Each coefficient carries two quantities. Its magnitude $|X(f, t)|$ is how much energy the signal has in that frequency band at that time, and its phase $\angle X(f, t)$ is where in repetition of the waveform the signal is at that time. The magnitude is what the model uses to determine which parts of the mixture belong to the target speaker, and the phase is what the model uses to reconstruct the waveform from the modified magnitude. The complex spectrogram is kept separately, since the mask of Section 3.2.7 is applied to it rather than to the network’s internal features.

3.2.3. Band plan

The spectrogram of Section 3.2.2 has 257 outputted frequency bins, each covering an equal slice of 31.25 Hz. Treating all 257 alike would waste the model’s capacity, because speech does not spread itself evenly across them. Most of the energy, and nearly all of the structure that distinguishes one voice from another, sits at the lower frequencies: the pitch of a voice and the resonances that give a vowel its identity all fall below about 4 kHz, and energy falls away steadily above that. The bins near the top carry far less information and thus would be wasteful to model with the same level of detail as the bins near the bottom.

A band plan is the decision of how to group those bins. Bins are gathered into contiguous groups, called bands, and each band is then handled separately by the network: it gets its own normalisation and its own projection, so a band is compared against itself over time rather than against the loud low-frequency bands next to it. Grouping few bins per band keeps fine detail. Secondly grouping many bins into one band summarises coarsely but costs far less. A band plan therefore decides where the model’s resolution is spent.

The plan in this work is 32 bands over the 257 bins: fifteen bands of 100 Hz covering 0 kHz to 1.5 kHz, ten of 200 Hz covering 1.5 kHz to 3.5 kHz, five of 500 Hz covering 3.5 kHz to 6 kHz, and the remaining 6 kHz to 8 kHz as one band. Figure 3.4 shows it against a real mixture in

the training set. The effect is a factor of twenty between the finest and coarsest resolution. This plan follows the implementation from Zhang et al. [20] for their 16 kHz target speaker extraction model and used by the challenge baseline built on it, but no reason is given there for these particular boundaries, and no explanation is given in previous papers. The original band splitting was designed for music [7], where the sources are instruments rather than voices. It is adopted here as the baseline so that this work’s results are comparable with that lineage, and it is recorded as an unjustified inheritance rather than a motivated choice.

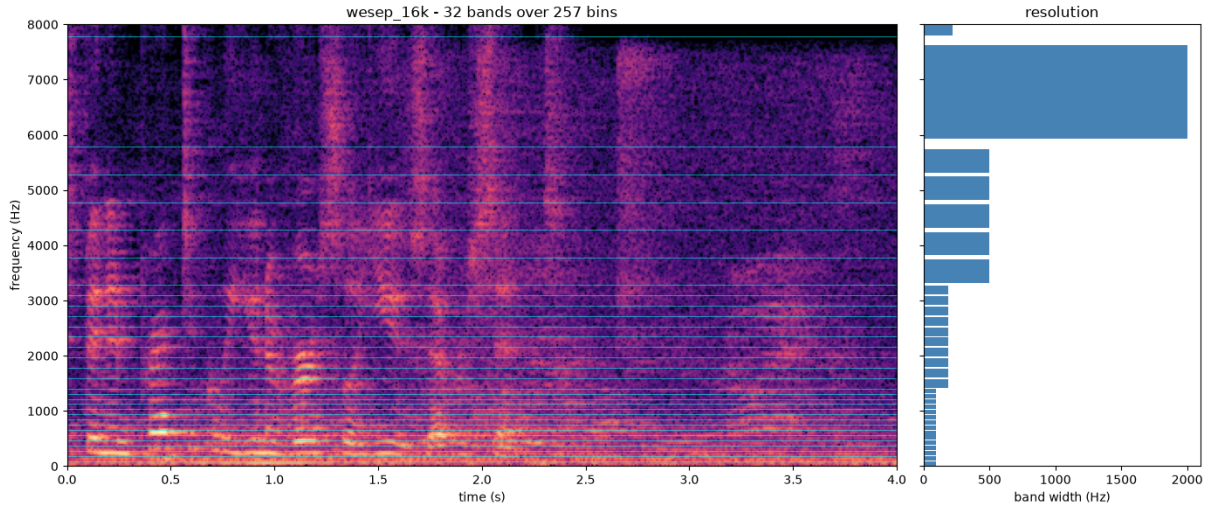


Figure 3.4: The band plan used here, shown against a four-second mixture from the training set. Left: the band boundaries drawn over the spectrogram, closely spaced at the bottom where the speech structure is and widening upwards. Right: the width of each band in hertz. The lowest fifteen bands are 100 Hz wide; the single highest spans 2 kHz. This band plan follows the one used by Zhang et al. [20] for their 16 kHz model.

Why the spectrum stops at 8 kHz. The audio in this work is sampled at 16 kHz, meaning 16,000 amplitude measurements per second. When measuring a wave you need at least two measurements per cycle, one for the peak and one for the trough of the wave. Therefore the fastest possible wave that 16 kHz sampling can represent will complete 8,000 cycles per second. Anything faster cannot be distinguished from a slower wave passing through the same measured points. Therefore, the largest possible range that can be represented by the 257 bins have to span 0 kHz to 8 kHz rather than 0 kHz to 16 kHz.

3.2.4. Per-band normalisation

The bands defined by Section 3.2.3 are not comparable with one another. Speech energy falls away by roughly 6 dB per octave, so the lowest bands carry orders of magnitude more energy than the highest. Therefore, if all 257 bins were scaled together, there would be a severe imbalance where the high bands would be underrepresented. This would then hurt the performance of the model as the high bands would be diluted and would contribute almost

nothing to the gradient, and the model would in effect only learn to separate the bottom of the spectrum. Giving each band its own scaling is what prevents that. A band at 6 kHz is then judged against its own typical level rather than against the bands beneath it.

This module does two things to each band independently, following [8]. First it normalises the band, using a layer normalisation applied across the channels of that band. Secondly, because the bands must be the same size before they can be stacked together, a projection is required to map the band to a fixed width. Since there are varying sized and independent bands, each band has its own projection, which is a learnable linear layer that maps the band to a fixed width of 128 values. Every band is given the same structure but its own weights, so nothing is shared between bands and each is free to learn a representation suited to its own frequency range. The result is a stack of 32 bands, each with 128 features, which is then passed to the next module.

Normalisation is applied across the values of a band *within a single frame*. Each frame is therefore normalised using only itself. This is a deliberate design choice that opposes Yu et al. [8], who use batch normalisation for their online configuration. The opposition is for the following reasons: (i) although batch normalisation will work for online models as indicated by Yu et al. and is causal, there are two different operations of batch normalisation: training and inference. During training, the mean and variance are computed from the current batch, but during inference, these statistics are computed from the entire training set. The function that is optimised during training is therefore not the function that is deployed. (ii) That gap is unusually wide here. A batch is twelve four-second crops drawn at random from a corpus spanning a range of reverberation times, signal-to-interference ratios and noise levels, and in which roughly three crops in ten contain no target speech at all. The statistics of a batch therefore depend on what was drawn, and a silent crop normalised alongside loud ones is not normalised as it would be at inference. Because normalisation is per band, the effect is worst in the high bands, where the energy is smallest and the variation between batches is largest relative to the signal. (iii) Single frame normalisation carries no state and is unaffected by the amount of audio processed, so a four-second training chunk is normalised exactly as a stream that has been running for a minute.

3.2.5. Residual LSTM block

The idea of the residual LSTM block is to model the sequential structure of speech such that the model can leverage patterns that unfold over time. There are two kinds of sequential structures that are modelled here. The first is the sequence over time, which is the natural order of speech. The second is the sequence over frequency. Think of the time being unidirectional, as the speech only makes sense to unfold forward in time, but the frequency is bidirectional, as all of the bands of a frame are available at the same instant and can be traversed in either direction. The block is used for both kinds of modelling, with the same structure but different parameters.

The recurrent layer is what carries information forward where this work leverages a long

short-term memory layer, or LSTM. This is a recurrent layer with an internal mechanism for deciding what to keep in that summary and what to discard, which is what allows it to retain information over hundreds of frames rather than only the last few. This structure also suits the deployment target directly. At inference time, since the model is running in a streaming fashion, the LSTM's internal state is preserved between frames and information can be carried forward indefinitely. The LSTM is therefore a natural fit for the task, and it is used in the same way for both the time and frequency sequences.

The block is made of three steps applied in order, following [7]. The input is first normalised, then passed through the LSTM, and the LSTM's output is then mapped back to the input's width by a learned projection. The final addition is a residual connection and is inspired by the original ResNet [21] architecture. This allows the gradient an unobstructed path back through the computation stack of all the blocks, which matters because six of these blocks are stacked in Section 3.2.6. This is important as the LSTM is a deep recurrent layer, and the gradient can vanish or explode as it passes through the LSTM's internal state. The residual connection allows the gradient to bypass the LSTM entirely, which is what allows the model to train when the gradient vanishes within the LSTM.

What is used here. The LSTM has a single layer with a width of 192, taken from Yu et al. [8], whose 16 kHz model specifies that width. This is deliberately narrower than the reference implementation of the same architecture, which uses twice the feature width, or 256. Because the LSTM's width differs from the 128 features entering the block, the projection is not optional bookkeeping: it is what makes the addition dimensionally possible at all.

The block is written to be indifferent to what its sequence axis represents. It receives a batch of sequences and processes them; whether those sequences run along time or along frequency is decided by the caller. This is what allows the same block to be used for both kinds of modelling in Section 3.2.6, with one critical difference between the two uses. Along time the LSTM runs forward only, because a backward pass over time reads frames that have not yet arrived. Along frequency it runs in both directions, which costs nothing in latency, since all the bands of a frame are available at the same instant.

3.2.6. Band and sequence model

The band and sequence model is the implementation of the residual LSTM block of Section 3.2.5 in a stack of six blocks, using the input from Section 3.2.4. After the per-band normalisation and projection, there are 32 bands of 128 features each. The model is now tasked for the two kinds of sequential modelling with the residual LSTM blocks as previously mentioned: time and frequency.

Each of these blocks is designed to make two passes. The design is to first make a pass over the time dimension (unidirectional), and then make a pass over the frequency dimension (bidirectional). This is considered as follows:

- **Time:** The recurrence goes forward only, as a backward pass would assume knowledge of future frames that have not yet arrived.
- **Frequency:** The recurrence goes over the bands of a frame in both directions. This pass is what lets the bands inform one another: a voice lays down a harmonic pattern across many bands at the same instant, so whether a given band belongs to the target is often not decidable from that band alone. Frequency itself is not a causal dimension. This can be reasoned by the fact that all the bands of a frame are all available simultaneously so the bidirectional recurrence does not introduce any latency.

Pairing the two passes in a single block is the design of Luo and Yu [7], who motivate it by analogy with dual-path recurrent networks for speech separation rather than by an argument specific to bands. Stacking six such blocks follows Yu et al. [8], where their 16 kHz model uses that specific number of blocks. The effect of stacking the blocks is to allow for the accumulation of evidence over longer time spans. The evidence can then be shared across bands of the next block and so forth.

3.2.7. Estimation module

The estimation module is responsible for producing an output spectrogram that contains the estimated target speaker’s speech. This is done by creating an output network per band to reconstruct the target speaker’s speech from the features produced by the band and sequence model. However, there are two methods to do this: either to predict the output spectrogram directly, or to predict a mask that is applied to the mixture. The latter is the approach taken here, following Yu et al. [8].

Yu et al. realised that the mixture audio itself already contains the target’s speech, and thus it would be more efficient to mask the mixture rather than predict the output spectrogram directly. This mask is a number for each time-frequency bin, which the mixture is multiplied by.

A mask alone is not enough however as multiplication can only ever scale what a bin already holds. In the cases where the interferer completely dominates a bin no multiplier recovers the target as the multiplication operation would not be able to scale the drowned out target. To solve this, the estimation module makes use of a residual spectrogram and is predicted directly and appended. Yu et al. [8] introduce it for exactly this reason, to absorb what the mask alone produces as artefacts. It is useful to not bound this mask and is what the original implementation encouraged. The estimate for a band is then

$$\hat{S} = M \otimes X + R, \quad (3.2)$$

where $\otimes$ is the complex product, taken bin by bin. The X that the mask multiplies is the original complex mixture and is taken as is from the band split and not from the output of the model’s

features. The features are used only to *decide* how to mask the input. The conditioning feature of Section 3.2.8 therefore never enters the signal path either.

Each band has its own two-layer output network: a normalisation, then a projection to a width of 384 with a $\tanh$ activation. Finally we have two output layers reading from it, one producing the mask and one the residual. The width of 384 is the value Yu et al. [8] specify.

The mask output layer has to output four pairs of real and imaginary parts for each frequency bin. Two of them are taken as the mask values, and the other two are passed through a sigmoid, giving a number between zero and one that each mask value is then multiplied by. This arrangement is a gated linear unit [22], and it is what both source papers specify for this layer [7, 8].

3.2.8. Time-frequency map

The TF Map is the enrollment-conditioning module to turn normal speech enhancement into the TSE task. The idea behind this module is for the model to take an enrollment of the target speaker, and use the unique audio signature of that speaker to condition the model to extract only that speaker from the mixture. The general process includes processing the enrollment clip through the STFT module, extracting the magnitude of the enrollment, and then breaking up the spectrogram into basis vectors. These basis vectors are then compared to the mixture’s spectrogram using cosine similarity.

Concretely, let $|X| \in \mathbb{R}^{F \times T}$ be the magnitude spectrogram of the mixture and $|X_e| \in \mathbb{R}^{F \times T_e}$ that of the enrollment, and write $|X_t|$ and $|X_{e,j}|$ for one frame of each. The module proceeds in three steps. *First*, every frame of both spectrograms is scaled to unit length,

$$\tilde{x}_t = \frac{|X_t|}{\| |X_t| \|}, \quad \tilde{e}_j = \frac{|X_{e,j}|}{\| |X_{e,j}| \|}. \quad (3.3)$$

Second, each mixture frame is compared against every enrollment frame by cosine similarity. Because (3.3) has already removed the length of each frame, the cosine of the angle between two frames is simply their inner product:

$$C_{t,j} = \cos \theta_{t,j} = \frac{\langle |X_t|, |X_{e,j}| \rangle}{\| |X_t| \| \| |X_{e,j}| \|} = \langle \tilde{x}_t, \tilde{e}_j \rangle. \quad (3.4)$$

This is what makes the comparison of spectral *shape* rather than of loudness: a quiet frame and a loud frame with the same spectral pattern are identical under (3.4). Note also that magnitudes are non-negative, so $C_{t,j} \in [0, 1]$ for every pair.

Third, the similarities of a mixture frame are turned into weights over the enrollment, and

those weights are used to blend the enrollment frames,

$$A_{t,j} = \text{softmax}_j(C_{t,j}), \quad \mu_t = \sum_{j=1}^{T_e} A_{t,j} \tilde{e}_j, \quad (3.5)$$

following [20]. The softmax runs along the enrollment index j , so each row of A sums to one and describes how to blend the enrollment frames for one instant of the mixture. μ_t is a guessed spectrum: for each mixture frame, a weighted average of all the enrollment frames, with the weights given by A . It is a rough estimate of the shape of the target speaker's spectrum at that instant.

μ_t describes only that shape. It is built entirely from enrollment frames but on its own it says nothing about whether the target is speaking. Note also that the mixture's magnitude has not been used at any point so far: (3.3) removed it before the comparison in (3.4) was made. The final step is to address the energy component. The estimate is scaled to unit length and the mixture's spectrum is projected onto it which measures how much of the mixture's actual energy lies in the pattern the enrollment predicted:

$$u_t = \frac{\mu_t}{\|\mu_t\|}, \quad m_t = \langle |X_t|, u_t \rangle, \quad \phi_t = m_t u_t. \quad (3.6)$$

Where m_t is the scalar projection of the mixture frame onto the template of the target speaker and describes with a single number per frame how much of the frame's energy lies in the direction of the enrollment. ϕ_t is the final feature, which is the template scaled by that projection.

Where it enters and what it costs. The result is a quantity of the same shape as the spectrogram itself, one value per frequency bin per frame. It is therefore attached as a third input channel alongside the real and imaginary parts, before the band split, so that it is divided into bands and projected by the same machinery as the rest of the input. Nothing else in the architecture changes. Because the extra channel only widens the input of the per-band projections, the entire conditioning path costs roughly 33 000 parameters, under half a percent of the model, and it adds no learned parameters of its own at all: (3.3) through (3.6) contain nothing to train.

3.2.9. Objective function

The objective has to express two different requirements, because the training data contains two different situations. Where the target speaks, the output should match the target signal. Where the target is silent as the result of either the trial contains no target speech at all, or because the four-second crop happened to land in a gap between the target's utterances, there is no signal to match, and the correct output is silence. A single loss cannot serve both use cases and so the objective is therefore split, and each crop is scored by the branch that applies to it.

Throughout, x is the mixture, s the target reference and $\hat{s}$ the model's output, all in the time domain over one training crop.

Target present. The first term measures how much of the output is the target and how much is everything else, following [1]. Importantly, the loss needs to be invariable to the overall volume as if the $\hat{s}$ is of the correct shape, it should not be penalised for being loud or quiet. Therefore the first step is to find the single gain that best explains the output as a scaled copy of the target, which will be calculated as a projection,

$$\alpha = \frac{\langle \hat{s}, s \rangle}{\|s\|^2}, \quad s_{\text{proj}} = \alpha s, \quad (3.7)$$

and then comparing the energy of that scaled target against the energy of whatever the output contains beyond it. The loss is modelled after the Scale-Invariant Signal-to-Distortion Ratio (SI-SDR). The loss is therefore,

$$L_{\text{pres}} = -10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|\hat{s} - s_{\text{proj}}\|^2 + \tau_{\text{pres}} \|s_{\text{proj}}\|^2}. \quad (3.8)$$

Lower is better. The loss is secondly in terms of decibels (dB), which is where the logarithm comes from. The τ_{pres} term in the denominator is a floor. Without it a perfect output would score $-\infty$, and the gradient would keep rewarding refinement of crops that are already essentially solved. With $\tau_{\text{pres}} = 10^{-3}$ the best attainable score is -30 dB. This constant follows the suggestion from CARTSE [1]. The reasoning is that an output whose residual error already sits 30 dB below the target is close enough to be treated as solved, so it would be wasteful to keep spending gradient on it. Two properties of this measure should be stated plainly, because they are what the second term exists to compensate for. The first is that everything the output contains beyond the target is collected into one denominator. Residual interfering speech, residual background noise and artefacts the model itself introduced all count identically per unit of energy, although they are not equally damaging: a fragment of the interferer left in the output can be transcribed downstream as words the target never said, whereas a diffuse artefact of the same energy usually cannot. The measure has no way to express that difference. The second is that it is an energy ratio summed over the whole crop, so it is dominated by whatever is loudest in it. Speech energy sits overwhelmingly in the low frequencies and in vowels, which means a model can score well on (3.8) while reconstructing fricatives and the upper spectrum poorly. Those are the parts that carry consonant identity, and therefore much of what makes speech intelligible rather than merely present.

Multi-resolution spectral term. Equation (3.8) is a single number computed over the whole waveform, which makes it ignore the location of errors in the spectrum. Since it cannot express *where* in the spectrum the error lies, a model that reconstructs the loud low bands well and the quiet high bands poorly is barely penalised and secondly since being scale-invariant, it does not constrain the output’s absolute loudness at all. The idea is to use a loss that considers multiple resolutions of s and $\hat{s}$. By considering different granularity of the spectrogram, the model is

encouraged to reconstruct both the low and high frequency bands well. A spectral term allows for this following [8]:

$$L_{\text{MR}} = \frac{1}{|\mathcal{N}|} \sum_{N \in \mathcal{N}} \left(\left\| |S_N|^p - |\hat{S}_N|^p \right\|_1 + \left\| \Re S_N - \Re \hat{S}_N \right\|_1 + \left\| \Im S_N - \Im \hat{S}_N \right\|_1 \right), \quad (3.9)$$

where S_N and $\hat{S}_N$ are the spectrograms of the target and the output (Equation 3.1) window length N and hop $N/4$, and each norm is averaged over its time-frequency coefficients.

Three choices inside (3.9) are worth stating. The exponent $p = 0.3$ compresses magnitudes, which will highlight the quieter components relative to loud ones so that an error in a low-energy region is not diluted just because it is quite. Without the compression the term would be dominated by the same loud low bands that already dominate (3.8). The real and imaginary terms are left uncompressed and they are what make the term sensitive to phase: a magnitude-only loss would be indifferent to the phase that the complex mask of Section 3.2.7 is free to change. Finally the term is evaluated at several window lengths rather than one, because a single analysis window can hide errors at its own scale.

The window lengths used are 8, 16, 32 and 64 ms. This is with respect to the model's own 32 ms framing. The model builds its output by masking 32 ms frames and overlapping them, so the places that most errors would occur can be expected to exist at frame boundaries. The 8 ms and 16 ms windows resolve this and the 64 ms window catches harmonic structure that the short ones blur.

Target absent. When there is no target speaker present in the mixture audio, the model should output silence and suppress all other noise. The model should therefore penalise any output that is not silent. This is modelled after a modified version of the absolute level loss from CARTSE [1], where in this works the loss now contains the denominator term.

$$L_{\text{abs}} = 10 \log_{10} \frac{\|\hat{s}\|^2 + \tau_{\text{abs}} \|x\|^2}{\|x\|^2}. \quad (3.10)$$

Including the denominator term makes the loss relative to the input, which is directly readable: 0 dB means the output is as loud as the mixture, that is, the model did nothing; -10 dB means the interfering speech was suppressed by 10 dB; and the same τ_{abs} floors the term at -20 dB, beyond which the output counts as silent. A positive value means the model amplified the mixture, which is a direct failure. The inclusion of the denominator also allows for scale invariance, which puts the loss in the same units as the loss for the target present case.

Output level. L_{pres} is scale-invariant and L_{abs} rewards silence, leaving a gap in the training that would otherwise allow a model to simply mute its output. This gap is filled by the third term, which ensures that the model does the following: (i) prevents the indiscriminate output of silence, and (ii) ensures a reasonable level of output. It bounds the output level against the

target's, with a dead zone of $\pm\delta$:

$$L_{\text{gain}} = \max\left(0, \left| 20 \log_{10} \frac{\text{rms}(\hat{s})}{\text{rms}(s)} \right| - \delta\right), \quad \delta = 3 \text{ dB}. \quad (3.11)$$

Inside ± 3 dB of the target's level the term is exactly zero, where rms is the root mean square. Its weight, $w_g = 1.69$, is derived rather than chosen: it is the geometric midpoint of the weight at which muting stops paying (where what muting saves on (3.10) equals what it costs on (3.11), both weighted as in (3.12)) and the weight where output level will outweigh the actual content extracted.

Combining the two branches. Each term is averaged over the crops it applies to, and the two averages are weighted against one another:

$$L = (1 - w) \underset{\text{target present}}{\text{mean}} \left[L_{\text{pres}} + w_m L_{\text{MR}} + w_g L_{\text{gain}} \right] + w \underset{\text{target absent}}{\text{mean}} \left[L_{\text{abs}} \right]. \quad (3.12)$$

Averaging within each branch rather than over the batch as a whole matters more than it appears to. Under a single batch-wide mean, the share of the gradient belonging to the silent crops would be whatever fraction of the batch happened to be silent, so the balance between the two requirements would fluctuate from step to step on the luck of the draw. Under (3.12) that balance is fixed by w .

Which branch a crop belongs to is decided from the audio of the cropped target itself, not from the trial's label. A trial in which the target does speak can still yield a crop containing no target speech, and this occurs in 5.8% of crops. Routing such a crop to the present branch would put an all-zero reference into (3.7), where α becomes 0/0. the implementation asserts against this rather than relying on the labels agreeing.

Weights. The weight w on the silent branch is derived rather than chosen. Silent crops make up a measured 0.297 of the training crops and, following [1], each silent crop is counted twice ($\eta = 2$). The present branch then carries $1 - p = 0.703$ and the absent branch $\eta p = 0.594$, and normalising the two to sum to one gives

$$w = \frac{\eta p}{(1 - p) + \eta p} = \frac{0.594}{1.297} = 0.458.$$

The weight w_m on the spectral term is derived from the target present and absent losses. The reason for this term is equations (3.8) and (3.9) are in different units. L_{pres} is in units with respect to a ratio in decibels, and L_{MR} is the mean absolute difference between compressed magnitudes. Therefore the weighting term w_m will reconcile the two terms to be on the same scale. This is done by evaluating on the output that does nothing at all, $\hat{s} = x$, over 300 training crops which will simulate the scenario for the average losses for an untrained model. The median values for L_{pres} and L_{MR} are -5.91 and 0.184 respectively. The aim is for the spectral

term (Multi-Resolution) to be roughly 30 % of the present branch at the start of training. The calculation then becomes

$$w_m = \frac{0.3 \times |L_{\text{pres}}|}{L_{\text{MR}}} = \frac{0.3 \times 5.9088}{0.1842} = 9.62$$

Differences with the source objective. The present and absent terms are inspired from the CARTSE system [1], the online-track submission whose task most closely matches this one, but they are slightly modified. Table 3.1 lists the differences. The first is the most consequential: as published, the floor in (3.8) is taken on $\|s\|^2$ rather than on $\|s_{\text{proj}}\|^2$, which breaks the scale invariance the term is built on. With that form, an output of the correct shape scaled by a gain g scores better the louder it is made and it is not bounded. Flooring on the projection instead makes numerator and floor scale together, so the two cancel and the score is flat with respect to gain.

Table 3.1: Departures from the objective of [1] as published. Each is recorded with its reason in the project’s decision log.

	As published	As used here
1. Floor in (3.8)	On $\tau_{\text{pres}}\ s\ ^2$, which is not scale-invariant	On $\tau_{\text{pres}}\ s_{\text{proj}}\ ^2$, so that gain cancels
2. Scale of (3.10)	Unnormalised, so the value shifts with input loudness	Divided by $\ x\ ^2$, so 0 dB means “did nothing”
3. Averaging over the batch	One mean over all crops in the batch	A mean within each branch, so the balance is set by w alone
4. Number of weights	A separate weight on the silent term	Folded into w ; under branch-wise means the two are one degree of freedom
5. Silence weight	Twice the absent rate of the source corpus	$w = 0.458$, the same emphasis at this project’s measured 0.297
6. Windows in (3.9)	All at or above the model’s frame length	8, 16, 32 and 64 ms, slightly lower and higher than the model’s 32 ms frame length

3.3. Extension model architecture

3.3.1. Splitting the cue into parts

The baseline architecture passes the network the TF-Map cue of [20] and rescaled by an energy-recovery step as seen in WeSep implementation [23] in Equation 3.6. By Equation 3.3, every mixture frame is its loudness times its shape, $|X_t| = \| |X_t| \| \tilde{x}_t$. Substituting this into m_t of

Equation 3.6 shows what the baseline’s single number contains:

$$m_t = \langle |X_t|, u_t \rangle = \| |X_t| \| \langle \tilde{x}_t, u_t \rangle = \| |X_t| \| c_t, \quad (3.13)$$

where

$$c_t = \cos \theta_t = \langle \tilde{x}_t, u_t \rangle \quad (3.14)$$

is the cosine similarity of Equation 3.4 which is measured with the blended template of u_t instead of an enrollment frame. The problem with m_t in the baseline architecture is that once multiplying the loudness and the similarity into a scalar, the network cannot tell distinguish between a frame that is very loud but not similar to the target and a frame that is very similar but not loud. The idea is to pass the parts instead of the product. Two parts are already defined: the template direction u_t of Equation 3.6 and the similarity c_t of Equation 3.14. The third part is new and is the residual

$$r_t = |X_t| - m_t u_t \quad (3.15)$$

which is the part of the mixture frame that the template of the target does not explain. A property of the residual is that it is orthogonal to the template, shown by:

$$\langle r_t, u_t \rangle = \langle |X_t| - m_t u_t, u_t \rangle = \langle |X_t|, u_t \rangle - m_t \langle u_t, u_t \rangle = m_t - m_t = 0. \quad (3.16)$$

where the last equality uses the fact that u_t is a unit vector. The orthogonality thus allows a frame to be split exactly into the equation:

$$|X_t| = m_t u_t + r_t \quad (3.17)$$

Therefore we can rebuild ϕ_t exactly as:

$$\phi_t = c_t \| |X_t| \| u_t \quad (3.18)$$

The choice to split was this work’s own design choice, but was a suggestion made by AI model Claude [24] to diagnose the baseline’s inability to learn the target’s voice effectively.

3.3.2. Speaker context embedding

When considering Section 3.3.1 the cue is still only described by the enrollments spectra. Target speakers that have similar pitch and vocal characteristics thus produce similar templates. Measured on the baseline architecture, the extractor was able to select the right voice 71.8 % of the time when two speakers were of different gender, but only 52.6 % when they were the same gender. The idea for this extension is to incorporate further target speaker identity. The second extension is to add a speaker embedding for each target speaker. Each enrollment clip is passed through a frozen speaker recognition network, ECAPA-TDNN [25] using the

SpeechBrain checkpoint trained on VoxCeleb [26], to produce a 192-dimensional embedding. The embedding is then passed through a small learned layer to produce a 128-dimensional vector to be used in the extractor. ECAPA itself is large at 20.77 million parameters and hence would cause storage concerns if it were included in the extractor. However, the ECAPA network can be run once per enrollment clip and the resulting embedding can be stored for later use. This allows for the extractor to be served locally on a device without needing to have the full ECAPA network, only the stored embedding for the target speaker.

The WeSep toolkit [23], an established open-source implementation of the baseline architecture, supports two uses of the embedding. The first (which is the one used in this work) is to consider utterance-level embedding, and the second is to consider contextual embedding of Zhang et al. [20]. Although WeSep supports the contextual embedding, it was discarded as an option for this work because it recomputes the embedding for every frame of the mixture, which is computationally expensive.

3.4. Metrics

This section describes the evaluation metrics used to assess the performance of a trained target speaker extraction model. These metrics are where the main contribution of the work lies. Since the main task is intelligibility of the target speaker and not necessarily the *quality* of the extracted target speaker speech, the metrics that are designed in this section aim to reflect that objective. The metrics are designed for the purposes of using the BSRNN model in conjunction with a live speech-to-speech model, such as Gemini Live [2]. Such a model is not a transcriber: it consumes audio through a learned encoder trained over a far wider distribution than read speech, and it is prompted rather than decoded. The metrics are therefore meant to show how well the model will translate as part of a live conversational system, and hence are designed to be used in conjunction with that system.

3.4.1. Trial definitions

Each trial passed through the model at evaluation produces a 4-tuple $(y_x, y_{\hat{s}}, y_s, y_d)$ where y_x is the input mixture speech transcription, $y_{\hat{s}}$ is the estimated target speech transcription, y_s is the target speech transcription, and y_d is the interferer speech transcription. The subscripts follow the audio notation of Section 3.2.9: x is the mixture, s the target reference and $\hat{s}$ the model output, with d denoting the interfering speech and $y_{(\cdot)}$ the transcription of a signal. These transcriptions are created in two ways in this project: (1) by using an automatic speech recognition (ASR) system [27, 28] to transcribe the audio to act as a baseline transcriber, and (2) by making use of a live speech-to-speech model that accepts the audio and outputs a transcription which is prompted with a standard prompt to transcribe the audio found in Appendix A.1. The tuple therefore does not represent necessarily the true transcriptions, but rather what the model/transcriber produces.

3.4.2. Text normalisation

To standardise the transcriptions created by the different methods, a text normalisation process is applied to the audio transcriptions. Specifically, using the `whisper-normalizer` library [29], the `EnglishTextNormalizer` package, seen in Appendix C of [28], is used to output the transcriptions into lower case, remove punctuation, expand contractions, standardise the spelling of numbers, and remove any non-alphanumeric characters. This is done to ensure that the transcriptions are comparable across different methods of transcription.

3.4.3. Live-model Content Fidelity Word Error Rate

The Live-model Content Fidelity Word Error Rate (LCF-WER) is a metric based off of the arithmetic of the standard Word Error Rate (WER) metric, but applied to a live-model transcription of the audio instead of a standard transcription. It is explicitly labelled as such as the standard WER metric is referenced in the training procedure, and is not associated with the live-model transcription. The standard WER metric is defined as follows:

$$\text{WER} = \frac{S + D + I}{N} \quad (3.19)$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference. It is essentially a minimum edit-distance formula, where the number of edits required to change a reference text into the proposed text is divided by the number of words in the reference text. However, there is one flaw in this approach that is required for our metrics to highlight: the broad classes of substitutions, deletions, and insertions do not demonstrate the nuance of how these errors can occur. The LCF-WER is used to represent that fact that the classes of errors are specific to live model transcription.

3.4.4. Interferer Content Rate

The Interferer Content Rate (ICR) is the first metric used to highlight one particular nuance of the LCF-WER metric: a substitution can occur when the model hallucinates a word that is not present in the reference, or the substitution can occur with the model hearing the right word, but from the wrong speaker. The idea for the ICR metric is to highlight the latter case, where the model has heard the correct word, but from the wrong speaker. This will help determine if our model is following speakers, or making up words that are not present in the reference.

Content Words. The ICR metric needs to identify which words are likely to be words that are not filler words that a model can easily hallucinate. Let $\mathcal{C}(\cdot)$ map a normalised transcription to

the set of content words it contains,

$$\mathcal{C}(y) = \{ w \in \text{words}(y) : w \notin \mathcal{S} \}, \quad (3.20)$$

where $\mathcal{S}$ is a fixed stopwords list, taken from the NLTK English stopwords corpus [30]. Function words are removed because almost any two English sentences share them, so an overlap counted over these words would rather count as grammatical issues than content. These content words will be used to make interferer-exclusive bag of words of which will indicate which words the model picks up from the interfering speaker.

Interferer-exclusive content. Firstly, if both the target and interfer say the same word, it is not possible to know which speaker the model heard, thus these words are not considered unique to the interferer. Let $\mathcal{D}$ be the unique content words of the interferer that are not present in the target, and the words that will be considered as leaked from the interferer are the intersection of the model output, $y_{\hat{s}}$ and the interferer-exclusive content words, $\mathcal{D}$,

$$\mathcal{L}(y_{\hat{s}}) = \mathcal{C}(y_{\hat{s}}) \cap \mathcal{D}. \quad (3.21)$$

Write $\ell = |\mathcal{L}(y_{\hat{s}})|$ for the number of words leaked on a trial and $a = |\mathcal{D}|$ for the number of possible unique words to leak. If a trial has $a = 0$, then the trial is not considered in the ICR metric, as there are no unique words to leak.

The contribution that this paper makes is to turn the ICR metric into a rate, as sentences are varying lengths, and thus the amount of leaked words changes with the length of the sentence. The rate secondly helps to answer the following situation: should the model that is finally chosen leak fewer words frequently, or leak more words infrequently. The rating definition is aimed to answer that question, and is defined as follows:

The rate. The rate can be reported at several thresholds. For a set of trials $\mathcal{T}$, and a threshold k of leaked words in a trial, the ICR metric is defined as the fraction of trials in which the number of leaked words ℓ is greater than or equal to k ,

$$\text{ICR@}k = \frac{|\{i \in \mathcal{T}_k : \ell_i \geq k\}|}{|\mathcal{T}_k|}, \quad \mathcal{T}_k = \{i \in \mathcal{T} : a_i \geq k\}. \quad (3.22)$$

Lower percentages are better. The rate is used to suggest how many words that are heard by the model that overlap with the interferer such that it is not considered chance if the word was heard.

The eligible set $\mathcal{T}_k$ depends on k . If in a trial there is only one exclusive content word available, it is impossible to achieve ICR@2, so if we keep this trial in the denominator it would skew the results.

3.4.5. Fabrication Rate

The Fabrication Rate (FR) acts as the second part to the ICR metric, where the metric aims to highlight the case where the model has hallucinated a word that is neither present in the target nor the interferer. This metric is again only computable as the LibriSpeech [16] dataset contains the exact transcriptions of the audio, and thus it is possible to determine if a word is hallucinated. The FR metric is defined as follows: Over the set $\mathcal{T}$ of trials with a non-empty response, the statistic is the mean invented count per trial of the number of words in the model output that are not present in either the target or interferer,

$$\text{FR}_{\text{count}} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} |\mathcal{I}_i|, \quad (3.23)$$

where $\mathcal{I}_i$ is the set of words in the i -th trial's output that are not present in either the target or interferer and $\mathbb{I}$ is the indicator function for non-membership. The FR metric can also be reported with a threshold k of invented words to consider a trial as a fabrication. The k value is used to determine the number of required words to not be included in either the target or interferer to be considered a hallucination. A word that is mis-transcribed is not necessarily a hallucination, as it is possible that the model heard the word but did not transcribe it correctly. A hallucination is a word that is invented by the model as a by-product of the probabilistic generation process that is a plausible but fabricated word, and not necessarily a mis-transcription of a word that was actually heard [31].

$$\text{FR}@k = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \mathbb{I} \left[|\mathcal{I}_i| \geq k \right], \quad k = 2 \text{ by convention.} \quad (3.24)$$

3.4.6. Deep Noise Suppression Mean Opinion Score

The Deep Noise Suppression Mean Opinion Score (DNSMOS) is a model based instrument that is used to evaluate the quality of the audio output as if heard from a human listener. The instrument itself is a neural network that predicts what a collection of humans would have rated an audio clip on a scale of 1-5, where 1 is bad and 5 is excellent. In this work, two different DNSMOS protocols are used. Specifically ITU-T P.835 [32] and ITU-T P.808 [33], predicted by the corresponding DNSMOS models [12, 34]. The former returns three metrics namely:

- **SIG**: The quality of the speech in the audio clip.
- **BAK**: The background noise quality of the audio, which measures how much the background noise is affecting the speech in the audio.
- **OVRL**: The overall quality of the audio.

This protocol was used in the SLT 2026 REAL-TSE Challenge [3] to measure how good the different teams TSE models were, but they found that some of the entrants were finding ways

to game the metric by making the audio sound better to the model, and not necessarily perform actual target speaker extraction. The challenge organisers therefore made use of the latter protocol in their final evaluation, the P808, which is a single metric from a separately trained model to measure overall quality of the audio clip.

3.4.7. Signal to Distortion/Interference/Artefact Ratios

This section discusses the following ratios: Signal-to-Distortion Ratio (SDR), Signal-to-Interference Ratio (SIR), and Signal-to-Artefacts Ratio (SAR). The ratios are used in the following manner: SDR is used to measure how wrong the output audio is in total, and the SIR and SAR are used to measure in which direction the output audio is wrong. The SIR measures how much of real audio that is not target speech is present in the output audio such as an interfering speaker or background noise, and the SAR measures how much of the output audio is not real audio, such as artefacts from the model. The output is split by orthogonal projection [35] into the target part s_{proj} of Equation 3.7, the interference e_{int} explained by the other speaker d and the noise, and the artefact e_{art} explained by none of the sources, so that $\hat{s} = s_{\text{proj}} + e_{\text{int}} + e_{\text{art}}$:

$$\text{SDR} = 10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|e_{\text{int}} + e_{\text{art}}\|^2} \quad (3.25)$$

$$\text{SIR} = 10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|e_{\text{int}}\|^2} \quad (3.26)$$

$$\text{SAR} = 10 \log_{10} \frac{\|s_{\text{proj}} + e_{\text{int}}\|^2}{\|e_{\text{art}}\|^2} \quad (3.27)$$

which are all measured in decibels (dB) where the higher ratio means better extraction of the target from the other audio in question. When training the model, the Scale-Invariant Signal-to-Distortion Ratio (SI-SDR) introduced by Le Roux et al. [36] is used as the loss function which takes inspiration from the SDR metric and was defined earlier in equation 3.8.

3.4.8. Anchors

The following terms are used to describe the lower and upperbounds of the results section. The floor references passing the full mixture audio, unprocessed to a listener, and the ceiling references passing a clean target audio to the listener. Simply put the floor is the worst-case scenario and simulates what a listener would hear if the system did nothing. The ceiling system is the best possible outcome, where there is perfect separation and the listener hears only the target audio. The floor and ceiling systems are not real systems, but rather reference points for the listener to compare against. These anchors are measured by passing the respective audio through the listener without interference from the extractor model and then are evaluated with the same metrics.

4. Experiments

This section discusses the overall setup to conduct the experiments, including the evaluation protocol, and the experimental setup. The results of the experiments are presented in Section 5.

4.1. Training setup

4.1.1. Data Augmentation

The training data is augmented with varying strategies to prevent overfitting. The first strategy is to provide every trial twice per epoch where the target and interferer roles are swapped. This strategy requires that the mixture requires two different outputs where the only varying information is the enrollment clip. The second strategy is to rotate enrollment clips for a target speaker. Each speaker per trial rotates between 3 enrollment clips so that the model does not memorise a single enrollment to a speaker. The third strategy is to randomly perturb the noise and reverberation for each trial per epoch. Finally, in the spirit of stochastic methods, a single random 4 s crop is taken from a random position in the trial for each epoch.

4.1.2. Optimisation and hardware

During training the AdamW optimiser [37] was used with a base learning rate of 0.0005, weight decay of 0.0001 and gradient clipping at 5.0. A batch size of 3 was used with mixed precision training. The learning rate schedule for the baseline included halving the learning rate if every 3 epochs saw no improvement in the validation score for the baseline. For the extension however, the learning rate halved when the Whisper WER stops improving. The target-absent term in the loss, L_{abs} , is off for the first 2 epochs, as the model in the early epochs would just output silence as this term is easy to optimise. The model is trained on Kaggle’s Tesla T4 GPU for roughly 15 hours of training time. The random seed is set to 42 for reproducibility.

4.1.3. Model Selection

The final model to base the evaluation on cannot simply be chosen by the model with the lowest validation loss as a result of the proxy training objective. The loss (Section 3.2.9) is a proxy for the downstream task, where the approach in the proxy is to reward better separation. However, better separation does not guarantee better recognition (Section 2.3): the epoch with the lowest loss for the extension has the highest WER of the chosen checkpoints as seen in Figure 4.1. Secondly, the model cannot be optimised directly against the WER as it is not differentiable. Finally a consideration is that the live judge is also too slow to run after every epoch. The model is thus chosen as follows: Whisper (paragraph 4.2.1) during training will create a shortlist of best candidate checkpoints based on the WER of the validation set for the extension model. The baseline model however had the candidates chosen by the lowest validation loss. Then

candidates are then scored by the live judge (paragraph 4.2.1) where the lowest WER is chosen as the final model with ties broken by the fewer invented words.

4.1.4. Systems compared

This work evaluates 3 models against the floor and ceiling (Section 3.4.8): baseline BSRNN (implemented following REAL-TSE [3]; Section 3.2), the extension BSRNN with speaker context embedding (Section 3.3), and WeSep [23]. There are four main reasons to include WeSep for comparison: (i) the REAL-TSE challenge [3], which this work uses as a guideline, builds its baseline system with WeSep; (ii) the WeSep model has the same BSRNN backbone with TF-Map cue of [20] as this work’s baseline and extension, where the main architectural difference is WeSep’s use of contextual embeddings rather than the single embedding per enrollment clip used by the extension, as discussed in Section 3.3.2; (iii) WeSep is used to validate the dataset constructed in this work, as WeSep was trained on clean Libri2Mix [9] and thus has not heard any of the speakers in the evaluation sets, so its extraction demonstrates what an established model can recover from these mixtures; and (iv) WeSep provides a potential upper bound for extracting the target. The WeSep model itself is non-causal and almost four times the size of the baseline (27.2 M versus 7.19 M parameters), and thus shows what extraction can achieve when latency and model size are not a concern.

WeSep was not trained on data that contained noise or reverberation, whereas trials in this work contain both. Secondly, WeSep itself has a causal separator, however, the features are normalised over the entire clip before the separator. This setup means that every output frame depends on audio that has not arrived yet and thus the model cannot be streamed and is treated as an offline system. Finally, the results of WeSep are not comparable to the results found in the REAL-TSE challenge [3] due to varying data, metrics and protocols.

4.1.5. Training curves

Both models were trained with the same loss and the same settings, but differ in how long they were trained and in how the final model was chosen. The baseline was trained for 16 epochs and the extension for 28. Figure 4.1 shows how the validation scores of each model changed during training, with epochs counted from zero.

4.2. Evaluation setup

4.2.1. Listeners

Offline ASR model The offline transcriber is the faster-whisper implementation [27] of the Whisper [28] model, run with the large-v3-turbo checkpoint. This is Whisper large-v3 with its decoder pruned from 32 layers to 4 and then fine-tuned, giving 809 M parameters against the 1,550 M of large-v3 [38]. The large Whisper variants are a common choice for

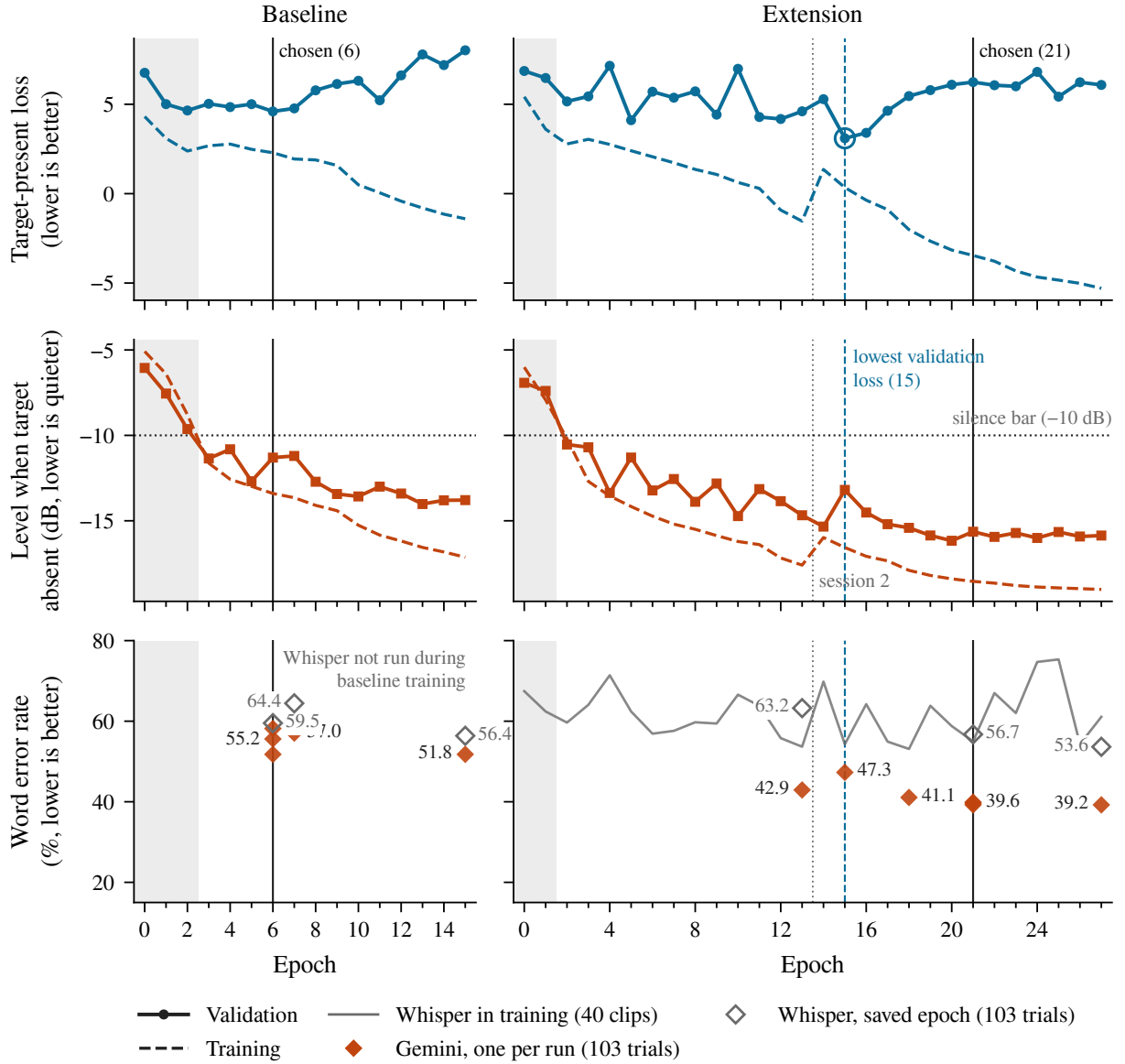


Figure 4.1: Training (dashed) and validation (solid) scores per epoch, counted from zero, for the baseline (left) and the extension (right). Top: target-present loss, the first bracket of Equation 3.12 (lower is better). Middle: output level on target-absent clips relative to the mixture, L_{abs} (lower is quieter); the dotted line is the -10 dB silence bar, and shaded epochs fail it. Bottom: word error rate on the 103 validation trials with both speakers present (lower is better). Each filled diamond is one run of the live judge on the same audio, labelled with the mean of the runs at that epoch; each hollow diamond is Whisper `small.en` scoring that saved epoch on the same 103 trials; the grey line is the in-training Whisper `small.en` check on 40 validation clips, which was not run for the baseline. The solid vertical line marks the chosen epoch, the blue dashed line the extension’s lowest validation loss, and the grey dotted line where the extension’s training resumed in a second session.

scoring how intelligible generated and extracted speech is [39, 40]. Decoding was greedy, with a single sampling temperature of zero and no conditioning on previously decoded text, so the same audio always produces the same transcript, and the language was fixed to English. The smaller `small.en` checkpoint (244 M parameters) is used only inside the training loop, where it transcribes a validation sample every epoch to shortlist checkpoints and schedule the learning rate. Scoring the results with a different checkpoint means the extension is not judged by the same listener it was tuned against.

Live judge The live listener is `gemini-3.7-flash` [2], which was connected to via the `google-generativeai` Python package through the Application Programming Interface (API). The judge, like the ASR model, does not separate anything. The Gemini model is prompted to transcribe the estimated audio from the extractor, and then transcribes what it is able to hear. The live model is the tool for which this work is being built for and so its result is the most important with respect to the overall evaluation of the extractor.

The prompt used can be found in Section A.1. It is deliberately worded to avoid instructing the judge to guess or pick a speaker to prevent the judge acting on the extractor’s behalf. Secondly, it does not instruct the judge to guess as this would be a form of hallucination, which is a metric that is being measured and thus should not be confounded by the prompt. Finally, since live models by default will return some kind of response, the judge is prompted to return a structured response of a status field alongside the transcript. The status field is used to indicate whether the judge heard speech or not, and if it did not hear speech, the transcript is left empty. This allows for a clear distinction for whether the model hallucinates as the status field can be used to determine whether the model heard speech or not.

Speech gating During experiments, it was common for both transcription tools to invent words when handed audio containing no speech frequently which led to many false positives. To address this issue, the Silero VAD v6.2.1 [41] speech gate is implemented to filter out any audio that appeared not to have speech that essentially acts as a pre-filter for the transcription tools. This method was considered for four reasons: (i) if the gate is applied to a mixture that contained speech and the gate determines that the estimated audio does not contain speech, then it is already an indication that the extractor has failed to extract the target speaker, (ii) if the gate is applied to a mixture that does not contain speech and the gate determines that the estimated audio does contain speech, then it is already an indication that the extractor has hallucinated, (iii) if the gate is applied to a mixture that does not contain speech and the gate determines that the estimated audio does not contain speech, then it is already an indication that the extractor has correctly determined that there is no speech, and (iv) this method is a practical implementation in a real system thus has a natural application to the evaluation of the extractor. Therefore it covers all possible scenarios and is a practical solution to the problem of hallucination. This method is supported by [42] and is applied for both the ASR and live judge.

4.2.2. Latency measures

During the course of experiments, the latency of the different systems was measured to compare the implemented model against (i) the latency specification for live transcription, and (ii) the latency of established systems. There are two main components to consider: time to consume and time to process the audio. The first component considers the real-time factor (RTF). The RTF is the ratio of the time taken to process the audio to the length of the audio, and for a model to be able to keep up with real-time audio, the RTF must be less than 1. This is The second component considers the latency created by the architecture itself which cannot be improved by faster hardware. To produce the output frame for a given window, the model must first have received that entire window. This forces a delay of the processing of the window as per the Section 3.2.2. For the model to be able to keep up with real-time audio, the RTF needs to be less than one, and secondly, the budget for processing a frame must be less than 200-300ms as a standard for live transcription. Finally, before either timing figure is interpreted, a probe checks whether a system can stream at all. A real-time factor cannot distinguish a model that is merely slow from one that requires the entire utterance before it can produce anything. The probe replaces all audio after a time T and measures how far the output *before* T moves: if changing the future changes what the model has already produced, then that output depends on audio which has not yet arrived, and the model cannot be streamed. For a strictly causal model the movement must be zero. The replacement is made at matched loudness so that the overall level of the input is preserved.

5. Results

[TODO:] Highlight that the model still has capacity to learn more with more training data and more parameters. Just that we needed a proof of concept. Also need to maybe put CARTSE's results in here for reference on another dataset.

5.1. Content fidelity

The main metrics to consider for this project is the performance of our extractor system with respect to the content fidelity on a live system. The content fidelity metrics are used to measure how the live artificial intelligence system is able to interpret the audio after the extraction process. The end goal is for the extractor to isolate and process the target speaker's audio in such a way for there to be minimal loss of content fidelity. Secondly, since the live system is a black box, non-deterministic system, the stability of the interpretation of the audio is also important to consider. The sections to follow consider the content fidelity for both a live judge and an offline transcriber.

5.1.1. The live judge - Gemini Live

The live judge results are shown in Table 5.1. These results are most indicative of the overall performance of the extractor system as the live judge is the end user of the extraction process. To better understand the variability of the live judge on the same audio, Figure 5.1 shows how the different systems changed in the LCF-WER metric across three independent runs.

Table 5.1: Content recovered by Gemini Live. gemini-3.7-flash, audio in and text out. Each value is the average of 3 runs on the same 103 trials, and $\pm$ is the standard deviation of those three run scores. Figure 5.1 shows how much the runs differ. Lower is better in every column.

system	LCF-WER ↓ (%)	ICR@2 ↓ (%)	mean leak ↓ (%)	invented ↓ /trial	FR@2 ↓ (%)
Floor	62.94 $\pm$ 0.37	75.73 $\pm$ 0.00	63.90 $\pm$ 0.65	1.27 $\pm$ 0.03	30.7 $\pm$ 3.1
Baseline	55.18 $\pm$ 3.22	61.49 $\pm$ 4.38	48.62 $\pm$ 3.27	1.91 $\pm$ 0.13	41.7 $\pm$ 3.4
Extension	39.59 $\pm$ 0.34	33.66 $\pm$ 2.02	23.08 $\pm$ 0.97	2.20 $\pm$ 0.11	46.9 $\pm$ 2.0
WeSep	25.80 $\pm$ 0.56	25.24 $\pm$ 0.97	13.04 $\pm$ 0.32	1.80 $\pm$ 0.02	38.6 $\pm$ 2.3
Ceiling	1.19 $\pm$ 0.18	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.22 $\pm$ 0.03	0.3 $\pm$ 0.6

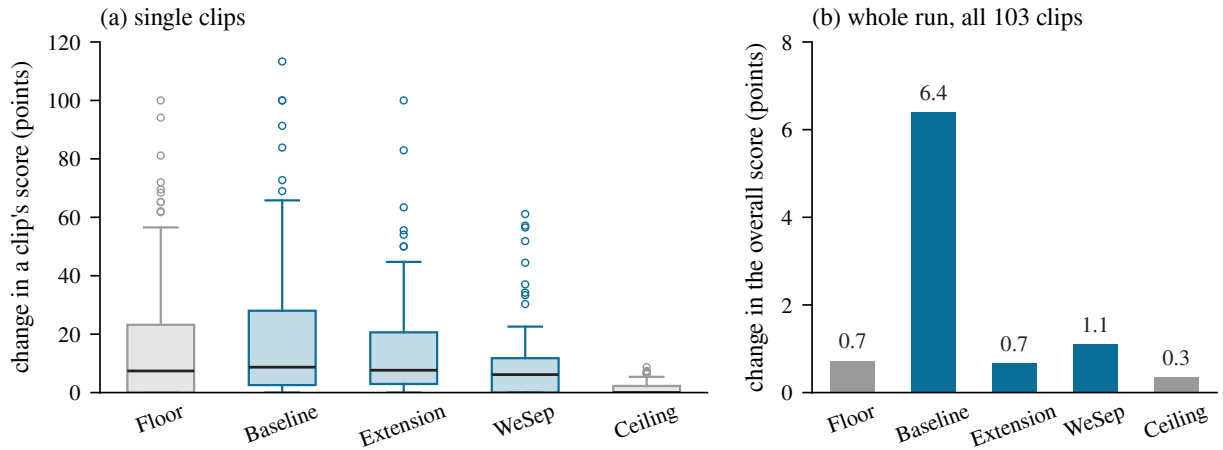


Figure 5.1: How much Gemini Live's LCF-WER changed across the three runs in Table 5.1, where change is the highest of the three run scores minus the lowest. (a) Is the box plot showing how the change varies per clip over the averaged runs. A clip's score can pass 100 % and can occur when Gemini adds words through hallucination or including interfering speech. (b) Change in the overall score over all 103 clips. Floor and ceiling (grey) are reference points.

The first comparison to be made is between the baseline and the floor. The baseline improves the LCF-WER by 7.76 points, which is a 12.3 % relative improvement over the floor. However, the baseline system evidently has extreme variation across the average performance over the three runs. The standard deviation of the LCF-WER is 3.22 points. There is a decisive improvement however in the baseline model's performance over the floor with regard to including an interfering speaker's content, as the average percentage of interfering words included in the output is 14.24 points lower than the floor. However, this came at a trade-off

with the number of hallucinated words. Compared against the floor, the baseline system has fabricated/invented more words on average per trial as seen in the table. Finally, the baseline system also more frequently fabricates indicated by FR@2. Therefore, not only is the baseline system resulting in more words in a trial being fabricated (which is indicative of high degrees of artefacts), but it is also more likely on a random trial to fabricate words than the just passing the raw mixture audio.

Overall, the baseline system is a clear improvement over the floor, which is indicative that the extractor has begun to find a signal in the mixture that is associated with the target speaker. The encouraging results can be seen when comparing the extension against the baseline and WeSep. WeSep acts as a benchmark for the best case scenario of what the BSRNN architecture can achieve from an extraction point of view although the WeSep model is both non-causal and cannot be used in a live system if there are latency concerns. For the extension, the inclusion of the pretrained ECAPA speaker encoder and passing the enrolment audio in parts instead of a single embedding has markedly improved the performance of the extractor. The LCF-WER has improved by 15.59 points over the baseline, which is a 28.3 % relative improvement. However, the most encouraging part of the results is how the extension has both reduced the number of leaked words per trial and the frequency of leakage by substantial amounts. The improvement can be attributed to the speaker specific information being passed to the extractor through the ECAPA speaker encoder. An interesting trade-off however is that the extension has increased the number of invented words per trial and the frequency of invention. This is indicative of the highly aggressive nature of the extractor, which was designed to mask out the interfering speaker's content however as a result, the masking can leave gaps or artefacts in the output which the live judge then interprets as words.

To get a better idea of how stable the live judge is on the same audio, Figure 5.1 demonstrates how the LCF-WER changed across the three independent runs but measured per clip instead of the overall average. The idea is to gain a better understanding of how the standard deviation of the LCF-WER is distributed across the clips. The first observation is the confirmation of the extreme variability of the live judge when passed extracted audio by the baseline. In fact, the baseline has worse variability than the floor and the highest average in change of performance between the three runs. A very encouraging result is that the extension seems to have the best stability even when compared to WeSep. On single clips, there are outlier performance changes of up to 100 points (when one run results in 100 more errors than another run), these outliers could be incredibly difficult cases that are not indicative of the overall performance of the system. However, as seen in the bar plot, the extension on average has the smallest changes in overall performance across the three runs.

Table 5.2: Gemini Live’s response in each trial case of Section 3.1.3. Cases 1 and 2, where the target spoke, report LCF-WER. Case 3, where only the other speaker spoke, reports the words Gemini output per trial, where 0 is correct, and how many outputs were silenced. Case 4 (noise only, $n = 8$) is omitted: no system produced a word, and every output was below the speech gate. The speech gate is part of the evaluation and is applied identically to every system: a clip with under 0.1 s of detected speech is not sent to Gemini and is scored as an empty response, and “silenced” counts those clips. Case 1 is the 3-run mean of Table 5.1; Cases 2 and 3 are one run each. Target alone is silent in Case 3 by construction.

	Case 1: both	Case 2: target only	Case 3: other speaker only	
	LCF-WER ↓	LCF-WER ↓	words/trial ↓	silenced ↑
system	(%, $n = 103$)	(%, $n = 47$)	($n = 42$)	(of 42)
Floor	62.94	1.65	30.69	0
Baseline	55.18	4.00	25.26	7
Extension	39.59	3.56	16.50	18
WeSep	25.80	5.40	17.95	7
Ceiling	1.19	1.46	—	—

Table 5.2 highlights where the extension shines. For the cases where the extractor needs to either suppress only background noise surrounding a target or only interfering speech, the extension performs better than WeSep. Secondly, if the extractor needs to suppress background noise such that a voice activity detector (VAD) does not detect any speech, the extension is able to do this more often than WeSep getting 18 noisy clips suppressed as compared against WeSep’s 7. On a larger set ($n = 123$), the extension passed 57 % of the other speaker’s words to Gemini against WeSep’s 70 % (a difference of -3.6 words per trial), and fell fully silent on 56 clips against WeSep’s 11. The improvement in the extension’s performance can be attributed to the diversity of training examples that include those specific cases. Interestingly, all the models perform worse on the target only case than just leaving the target unprocessed. This is likely due to the robustness of Gemini to background noise, leaving the extractor model to introduce artefacts that are misinterpreted by Gemini. Overall, the extension has shown improvement over WeSep in different acoustic scenarios, although still falls short when considering pure speech extraction with multiple speakers.

5.1.2. The offline transcriber

The offline transcriber serves to demonstrate how the extractor system performs on a standard transcription tool. The results are shown in Table 5.3. The offline transcription system seems like a natural alternative to the live system whilst building the architecture as the live systems make use of paid APIs and are not open source.

Table 5.3: Content recovered by the offline transcriber, faster-whisper large-v3-turbo. Lower is better in every column.

system	WER ↓ (%)	ICR@2 ↓ (%)	mean leak ↓ (%)	invented ↓ /trial	FR@2 ↓ (%)
Floor	63.04	76.70	62.78	1.59	38.83
Baseline	52.71	62.14	45.73	1.96	49.02
Extension	41.01	36.89	19.55	2.46	57.43
WeSep	29.13	22.33	12.46	2.02	54.37
Ceiling	1.98	0.00	0.00	0.45	8.74

The offline transcriber results produce the same conclusions as the live judge, however are obtained differently. The offline transcriber seems to suffer more with fabrication and performs worse across both the invented words per trial and the frequency of invention in comparison to the live judge. However, with respect to the interfering speaker’s content, the offline transcriber seems to be more susceptible in a trial to include interfering speech, however, the actual amount of interfering speech included in the output is lower than the live judge. Therefore the trade-off is that the offline transcriber will more likely make a mistake on a trial, but the mistakes it makes are less severe than the live judge. Overall, the finding still holds that the extension has improved the content fidelity of the target speaker’s audio over the baseline and is approaching the performance of WeSep. The offline transcriber however suggests that the gap between the extension and WeSep is smaller than what the live judge suggests, as the offline transcriber has an 11.88 point gap in LCF-WER between the two systems, whereas the live judge has a 13.79 point gap.

5.2. Error composition

The LCF-WER metric is a combination of three different types of errors to form the word error rate: substitution, deletion and insertion. The metric is useful for breaking down the weaknesses of the extractor system and to better understand how the system is introducing error. For example, a system with high deletions is indicative of over suppression of the target speaker’s content, a system with high insertions is indicative of hallucination or leakage, and finally a system with high substitutions is indicative of misinterpretation of the target speaker’s content. The error composition for both the live judge and the offline transcriber are shown in Table 5.4.

Table 5.4: Substitution (S), deletion (D) and insertion (I) rates for each listener. Lower is better.

system	Gemini Live			Offline ASR		
	$S \downarrow$	$D \downarrow$	$I \downarrow$	$S \downarrow$	$D \downarrow$	$I \downarrow$
Floor	29.22	5.97	28.08	33.41	6.93	22.70
Baseline	25.64	3.00	26.95	27.24	5.70	19.76
Extension	16.96	9.44	13.19	19.24	12.28	9.49
WeSep	10.62	6.78	8.99	13.45	5.91	9.78
Ceiling	0.87	0.15	0.03	1.60	0.15	0.23

The first observation is related to the live judge and how it performs when given a raw mixture. As expected, the floor system has a high insertion rate of 28.08 % which is indicative of the interfering speaker’s content being included in the output and then a high substitution rate of 29.22 % which is indicative of the live judge misinterpreting the target speaker’s content. The deletion rate is comparatively much lower at 5.97 % but not 0 %. This means that even with no masking, the live system can miss some of the target speaker’s content.

5.3. Signal-domain separation

A key part of the training process is to introduce the idea of separation. The three kinds considered in this project is separation of the target from the background noise (distortion), interference from other speakers (interference) and finally from artefacts introduced by the extractor system (artefact). The signal-domain separation results are shown in Table 5.5. These metrics highlight what the extractor system is doing in the signal domain and can best be used to understand the trade-offs between the different types of separation. A note about the values in the table: SAR for the floor and ceiling are both at +30 dB, as since these systems do not contain a model there can be no artefacts introduced.

Table 5.5: Signal-domain decomposition, in dB; higher is better. The ceiling reaches 30 dB by construction, where the cap created by the denominator floor $\tau = 10^{-3}$, and is not a measurement. Reference Section 5.2 for more details.

system	SDR $\uparrow$	SIR $\uparrow$	SAR $\uparrow$
Floor	-1.12	-1.12	+30.00
Baseline	+1.29	+3.66	+11.11
Extension	+2.29	+9.17	+6.28
WeSep	+4.68	+10.34	+8.26
Ceiling	+30.00	+30.00	+30.00

The most noticeable observation is that the extension has significantly improved the SIR metric over the baseline, and is comparable to the WeSep model. The gap in performance

between the extension and WeSep therefore seems to lie in the SAR and SDR metric. Therefore when considering how to improve the extension, the next iteration should potentially target mechanisms that reduce the artefacts introduced by the extractor system. Interestingly, the baseline has a highest model SAR metric. However this should not be attributed to the model being better at reducing artefacts, but rather due to the simplistic nature of the model design. The baseline model is not as aggressive in masking out the interfering speaker’s content, and therefore does not introduce as many artefacts into the output.

5.4. Perceptual quality

The perceptual quality metrics are used to indicate that extraction audio that sounds good to a proxy of a human listener does not necessarily translate to better content fidelity. The perceptual quality metrics are shown in Table 5.6.

Table 5.6: DNSMOS, personalised model. All scores are between 1 and 5 where the higher the score, the better.

system	P808 ↑	SIG ↑	BAK ↑	OVRL ↑
Floor	2.913	4.090	2.031	2.497
Baseline	2.955	3.233	2.286	2.183
Extension	3.030	2.957	3.045	2.306
WeSep	3.010	3.450	2.640	2.510
Ceiling	3.550	4.175	3.592	3.429

First considering P808 in isolation, the extension seemingly has a similar, if not slightly better, overall quality score than WeSep. Therefore by the one perceptual quality metric our extension produces audio that is more pleasing to a human listener (by the P808 model standards) than the WeSep model. However, when considering the other three metrics that form part of the second perceptual quality metric, the WeSep model has a better SIG and OVRL score than the extension. However, something fascinating to note is how the extension scores lower than the floor for both the SIG and OVRL metrics, and yet as seen in the content fidelity results, the extension has improved the content fidelity over the floor by a significant margin. This is an interesting observation as it demonstrates that the perceptual quality metrics are not good indicators for the purposes of this research project, which is to improve the content fidelity of the target speaker’s audio. The metric in itself is subjective to the actual perceptual system and does not provide indication that what sounds good to a human listener is necessarily better for the content fidelity of the target speaker’s audio.

5.5. Latency and throughput

Table 5.7: Latency and throughput over 2,250 chunks of 80 ms on CPU (i5-1135G7, 4 threads), except where marked. Baseline and extension figures are the median of three runs each, alternated in one session on an otherwise idle machine, with the range in brackets; WeSep is a single earlier run. The requirement is $\text{RTF} < 1$ and latency within 200–300 ms; the first is the binding constraint, because a chunk that finishes inside its own duration meets the latency budget automatically. Every figure is an *estimate* and not a streaming measurement: chunks are processed independently.

system	RTF ↓		latency (ms) ↓		meets
	mean	p99	mean	p99	budget
Baseline	0.68 (0.66–0.70)	0.79 (0.79–0.86)	175 (173–176)	183 (183–189)	yes
Extension	0.69 (0.68–0.71)	0.78 (0.77–0.82)	175 (175–177)	182 (182–185)	yes
WeSep	2.854	5.300	348.3	544.0	no [‡]

The latency and throughput results are shown in Table 5.7. The baseline and the extension model is able to meet the real-time factor and latency requirements for a live system. The WeSep model is not able to meet the requirements and therefore cannot be used in a live system. Overall, the extension is comfortably within the latency requirements and is able to process the audio in real-time. Furthermore, in the experimentation, the extension model did not miss any chunks and the worse case delay was 196 milliseconds, which is well within the 200–300 millisecond requirement.

6. Conclusion

6.1. Summary of contributions

6.2. Limitations and future work

The shortcomings of the work are the extractor’s weakness in removing interfering speech, whilst simultaneously not introducing artefacts that get misinterpreted by the downstream model as target speech.

References

- [1] L. Li and S. Seki, “CARTSE submission to the REAL-TSE challenge track 1: Online target speaker extraction,” CyberAgent, Inc., Tech. Rep., 2026, system description.
- [2] Google DeepMind, “Gemini Live API,” <https://ai.google.dev/gemini-api/docs/live>, 2026, accessed: 2026-08-31. Exact model identifier to be recorded with each result.

- [3] S. Wang, Z. Qian, K. Zhang, J. Han, Z. Liu, X. Yu, H. Li, M. Delcroix, K. Yu, L. Xie, M. Li, and H. Li, “SLT 2026 REAL-TSE challenge: Real-world target speaker extraction from conversational recordings,” arXiv, Tech. Rep. arXiv:2607.15198, 2026.
- [4] E. C. Cherry, “Some experiments on the recognition of speech, with one and with two ears,” *The Journal of the Acoustical Society of America*, vol. 25, no. 5, pp. 975–979, 1953.
- [5] K. Žmolíková, M. Delcroix, T. Ochiai, K. Kinoshita, J. Černocký, and D. Yu, “Neural target speech extraction: An overview,” *IEEE Signal Processing Magazine*, vol. 40, no. 3, pp. 8–29, 2023, also arXiv:2301.13341.
- [6] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing ideal time–frequency magnitude masking for speech separation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, no. 8, pp. 1256–1266, 2019.
- [7] Y. Luo and J. Yu, “Music source separation with Band-Split RNN,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1893–1901, 2023.
- [8] J. Yu, H. Chen, Y. Luo, R. Gu, and C. Weng, “High fidelity speech enhancement with Band-split RNN,” in *Proc. Interspeech 2023*, 2023, pp. 2483–2487.
- [9] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, “LibriMix: An open-source dataset for generalizable speech separation,” arXiv:2005.11262, 2020, reference only, never run here. Cited for its level-setting convention, which this project follows.
- [10] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, “Deep clustering: Discriminative embeddings for segmentation and separation,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016, pp. 31–35.
- [11] W. Zhang, J. Shi, C. Li, S. Watanabe, and Y. Qian, “Closing the gap between time-domain multi-channel speech enhancement on real and simulation conditions,” in *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2021, pp. 146–150, also arXiv:2110.14139.
- [12] C. K. A. Reddy, V. Gopal, and R. Cutler, “DNSMOS: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 6493–6497.
- [13] K. Iwamoto, T. Ochiai, M. Delcroix, R. Ikeshita, H. Sato, S. Araki, and S. Katagiri, “How bad are artifacts?: Analyzing the impact of speech enhancement errors on ASR,” in *Proceedings of Interspeech*, 2022, pp. 5418–5422.

- [14] P. Manakul, W. H. Gan, M. J. Ryan, A. S. Khan, W. Sirichotedumrong, K. Pipatanakul, W. Held, and D. Yang, “AudioJudge: Understanding what works in large audio model based speech evaluation,” in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 3644–3663, also arXiv:2507.12705 (2025).
- [15] R. F. Fela and P. Mowlaee, “Perceptually better, semantically worse: Measuring speech enhancement impact on LLM-based voice systems,” arXiv:2608.30348, 2026, accepted at EMNLP 2026.
- [16] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. South Brisbane, QLD, Australia: IEEE, 2015, pp. 5206–5210, source of every mixture in this work and of the starred references y^* . Chosen because it supplies both a clean per-speaker signal and exact ground-truth text, neither of which real conversational corpora provide.
- [17] G. Wichern, J. Antognini, M. Flynn, L. R. Zhu, E. McQuinn, D. Crow, E. Manilow, and J. Le Roux, “WHAM!: Extending speech separation to noisy environments,” in *Proceedings of Interspeech*, 2019, pp. 1368–1372, licensed CC BY-NC 4.0, which is the binding constraint on redistribution of every constructed mixture.
- [18] R. Scheibler, E. Bezzam, and I. Dokmanić, “Pyroomacoustics: A python package for audio room simulation and array processing algorithms,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 351–355, version 0.10.1. Shoebox room impulse responses by the image-source method, with a Sabine T60 feasibility check.
- [19] W. C. Sabine, *Collected Papers on Acoustics*. Cambridge, MA: Harvard University Press, 1922.
- [20] K. Zhang, J. Li, S. Wang, Y. Wei, Y. Wang, Y. Wang, and H. Li, “Multi-level speaker representation for target speaker extraction,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [22] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language modeling with gated convolutional networks,” in *Proc. 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 933–941.

- [23] S. Wang, K. Zhang, S. Lin, J. Li, X. Wang, M. Ge, J. Yu, Y. Qian, and H. Li, “WeSep: A scalable and flexible toolkit towards generalizable target speaker extraction,” in *Proc. Interspeech 2024*, 2024, pp. 4273–4277.
- [24] Anthropic, “Claude (opus 5 and opus 5.5), used via claude code,” <https://claude.ai>, 2026, accessed: 2026-09-21 to 2026-10-02.
- [25] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification,” in *Proc. Interspeech 2020*, 2020, also arXiv:2005.07143.
- [26] M. Ravanelli, T. Parcollet, P. Plantinga, A. Rouhe, S. Cornell *et al.*, “SpeechBrain: A general-purpose speech toolkit,” arXiv:2106.04624, 2021.
- [27] SYSTRAN, “faster-whisper: Faster Whisper transcription with CTranslate2,” <https://github.com/SYSTRAN/faster-whisper>, 2026, version 1.2.1. Accessed: 2026-08-31.
- [28] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518, also arXiv:2212.04356. The English text normaliser used in this work is defined in Appendix C.
- [29] whisper-normalizer contributors, “whisper-normalizer: Standalone text normalisation from OpenAI Whisper,” <https://pypi.org/project/whisper-normalizer/>, 2026, version 0.1.15. Implements the EnglishTextNormalizer of [28]. Accessed: 2026-08-31.
- [30] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, 2009, the English stopword list is taken from the NLTK stopwords corpus and snapshotted into this project’s source tree so the list cannot change with a library upgrade.
- [31] A. Koenecke, A. S. G. Choi, K. X. Mei, H. Schellmann, and M. Sloane, “Careless Whisper: Speech-to-text hallucination harms,” in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2024, pp. 1672–1681, also arXiv:2402.08021.
- [32] International Telecommunication Union, “Subjective test methodology for evaluating speech communication systems that include noise suppression algorithm,” ITU-T, Geneva, Recommendation P.835, Nov. 2003, rates the same clip three times on separate axes (SIG, BAK, OVRL) so a system cannot hide noise by damaging speech. Approved 11/2003 and not revised since.
- [33] —, “Subjective evaluation of speech quality with a crowdsourcing approach,” ITU-T, Geneva, Recommendation P.808, Jun. 2018, one overall rating per clip. A separate protocol

from P.835, not a summary of it. Revised 06/2021; the 2018 text is the one the DNSMOS papers follow.

- [34] C. K. A. Reddy, V. Gopal, and R. Cutler, “DNSMOS P.835: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 886–890, returns SIG, BAK and OVRL separately, per ITU-T P.835. OVRL is the score the REAL-TSE Challenge saw over-optimised.
- [35] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 4, pp. 1462–1469, 2006, source of the decomposition into interference and artefact ratios (SIR and SAR).
- [36] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, “SDR – half-baked or well done?” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 626–630.
- [37] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019, arXiv:1711.05101.
- [38] OpenAI, “whisper-large-v3-turbo,” <https://huggingface.co/openai/whisper-large-v3-turbo>, 2024, model card. Accessed: 2026-09-28.
- [39] P. Anastassiou *et al.*, “Seed-TTS: A family of high-quality versatile speech generation models,” arXiv:2406.02430, 2024, its Seed-TTS-eval benchmark scores English word error with Whisper large-v3.
- [40] H. Wang, J. Hai, D. Yang, C. Chen, K. Li, J. Peng, T. Thebaud, L. Moro-Velázquez, J. Villalba, and N. Dehak, “SoloSpeech: Enhancing intelligibility and quality in target speech extraction through a cascaded generative pipeline,” arXiv:2505.19314, 2025, scores word error with Whisper large-v3-turbo.
- [41] Silero Team, “Silero VAD: Pre-trained enterprise-grade voice activity detector,” <https://github.com/snakers4/silero-vad>, 2021, version 6.2.1, used unmodified. Defines what counts as speech for both the corpus index and the evaluation speech gate.
- [42] M. Barański, J. Jasiński, J. Bartolewska, S. Kacprzak, M. Witkowski, and K. Kowalczyk, “Investigation of Whisper ASR hallucinations induced by non-speech audio,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5, evaluates VAD gating against confidence thresholding and pattern matching; finds both give meaningful reductions and that no single method eliminates the problem. Also arXiv:2501.11378.

- [43] jiwer contributors, “jiwer: Evaluate automatic speech recognition,” <https://github.com/jitsi/jiwer>, 2026, version 4.0.0. Accessed: 2026-08-31.
- [44] International Telecommunication Union, “Algorithms to measure audio programme loudness and true-peak audio level,” ITU-R, Geneva, Recommendation BS.1770-4, 2015, integrated loudness, as implemented by pyloudnorm 0.2.0. All trial levels are set in these units.

A. Evaluation protocol details

A.1. Prompt for live-model transcription

Write down the words spoken in this audio, exactly as spoken.

If no words are audible, leave the transcript empty and set status to `no_speech`.

A.2. Pinned components of the measuring instrument

Component	Identity
Text normaliser	<code>whisper-normalizer 0.1.15</code> , <code>EnglishTextNormalizer</code> [28,29]
Baseline transcriber	<code>faster-whisper 1.2.1</code> , <code>large-v3-turbo</code> , int8 quantisation, greedy decoding [27,28,38]
In-loop transcriber	<code>faster-whisper 1.2.1</code> , <code>small.en</code> , int8 quantisation, greedy decoding; training-time checkpoint shortlisting only
Word error rate	<code>jiwer 4.0.0</code> [43]
Stopword list	NLTK English stopwords [30], snapshotted into the source tree so the list cannot change with a library upgrade
Judge model	<code>gemini-3.7-flash</code>
Random seed	42

Table A.1: Components of the evaluation instrument, pinned by version. A change to any row invalidates comparison with numbers reported before it.

B. Room construction

Each trial is rendered in its own simulated room rather than convolved with a recorded impulse response, so that the acoustics are a controllable variable rather than a property of whichever room happened to be recorded. Rooms are rectangular (“shoebox”) and the impulse responses are generated by the image-source method using `pyroomacoustics 0.10.1` [18].

For every trial the renderer draws the room’s three dimensions, a reverberation time, and independent positions for the microphone and the two talkers. Wall absorption is not drawn directly: it is solved for from the requested reverberation time and the room’s surface area by the Sabine relation, and the draw is rejected and retaken if the combination is not physically realisable. Both talkers are placed in the same room and share the microphone, so the two speech signals arrive with different but consistent room responses.

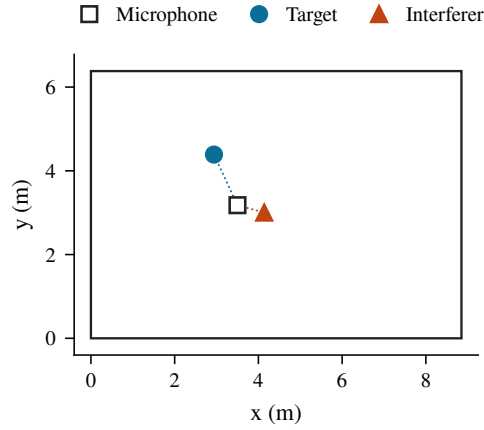


Figure B.1: Plan view of the room rendered for trial train-42-007495. The room measures $8.85 \times 6.38 \times 3.14$ m (177 m^3) with a reverberation time of 0.250 s, for which the Sabine relation gives a wall absorption of 0.5479. The microphone sits at (3.50, 3.18, 1.28) m, the target at (2.94, 4.39, 1.66) m and the interferer at (4.14, 3.00, 1.57) m, placing them 1.39 m and 0.72 m from the microphone respectively.

C. Construction parameters

Every parameter below is drawn independently per trial from the stated distribution, except where a regime overrides it. Ranges written $[a, b]$ are sampled uniformly on that interval.

Parameter	Distribution	Meaning
<i>Trial layout</i>		
Mixture length	$[15.0, 20.0]$ s	Duration of the rendered trial
Overlap ratio	$[0.0, 0.7]$	Fraction of target speech with the interferer also talking
Overlap tolerance	0.03	Accepted deviation from the requested overlap
Target activity ratio	$[0.15, 0.78]$	Fraction of the trial in which the target speaks
Activity tolerance	0.1	Accepted deviation from the requested activity
Utterances per source	≤ 6	Cap on how many utterances one recording may contribute
<i>Levels</i>		
SIR	$[-5.0, 15.0]$ dB	Target loudness relative to the interferer
SNR	$[0.0, 20.0]$ dB	Target loudness relative to the noise bed
Target loudness	$[-33.0, -25.0]$ LUFS	Absolute level of the target, BS.1770 integrated [44]
Clip ceiling	0.95	Peak after which the whole trial is rescaled together
<i>Room</i>		

Continued on the next page

Parameter	Distribution	Meaning
T_{60}	[0.25, 0.6] s	Reverberation time
Room length, width	[5.0, 10.0] m	Floor plan
Room height	[3.0, 4.0] m	
Source distance	[0.1, 2.0] m	Talker to microphone
Source height	[0.9, 1.8] m	Talker, seated to standing
Microphone height	[0.9, 1.8] m	
Early reflections	50 ms	Cutoff separating early echoes from the late tail
<i>Enrolment</i>		
Enrolment length	5.0 s	Single clip, levelled, carrying no room
EQ augmentation	$p = 0.5$	Three peaking biquads, log-uniform 120 Hz to 6400 Hz, ± 6 dB, $Q = 1.0$, RMS restored
<i>Speaker pairing</i>		
Same-gender fraction	0.5	Proportion of trials whose two talkers share a gender
<i>Speech detection</i> [41]		
Silence threshold	250 ms	Pause length after which a talker counts as stopped; this defines overlap
Speech probability	0.5	Frame-level threshold
Minimum speech	100 ms	Shortest run counted as speech
Speech padding	30 ms	Added either side of a detected run
Noise-bed rejection	0.5 s	Longest run of detected speech tolerated in a WHAM! clip
<i>Regimes</i>		
Regime weights	0.6 / 0.4	Proportion of trials drawn as base / hard
base SIR	[0.0, 12.0] dB	Regime override
base SNR	[8.0, 20.0] dB	Regime override
base activity	[0.45, 0.78]	Regime override
base T_{60}	[0.25, 0.5] s	Regime override

Table C.1: Construction parameters and their sampling distributions.